Aalto University
School of Science

Master's Programme in Computer, Communication and Information Sciences

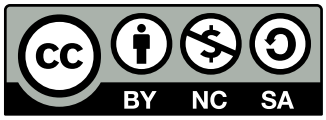
Context is king - Retrieval-Augmented Generation in enterprise-ready systems

Antti-Jussi Asunmaa

Master's Thesis
2026

© 2026

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 4.0 International" license.



Aalto University
School of Science

Author Antti-Jussi Asunmaa

Title Context is king - Retrieval-Augmented Generation in enterprise-ready systems

Degree programme Computer, Communication and Information Sciences

Major Computer Science

Supervisor Prof. Petri Vuorimaa

Advisor Lauri Salminen (Msc)

Collaborative partner Nordea Bank Oyj

Date 30 June 2026 **Number of pages** 61+13 **Language** English

Abstract

The abstract is a short description of the essential contents of the thesis, usually in one paragraph: what was studied and how and what were the main findings.

For a Finnish thesis, the abstract should be written in both Finnish and English; for a Swedish thesis, in Swedish and English. The abstracts for English theses written by Finnish or Swedish speakers should be written in English and either in Finnish or in Swedish, depending on the student's language of basic education. Students educated in languages other than Finnish or Swedish write the abstract only in English. Students may include a second or third abstract in their native language, if they wish.

The abstract text of this thesis is written on the readable abstract page as well as into the pdf file's metadata via the `\thesisabstract` macro (see comment in this TEX file above). Write here the text that goes onto the readable abstract page. You can have special characters, linebreaks, and paragraphs here. Otherwise, this abstract text must be identical to the metadata abstract text.

If your abstract does not contain special characters and it does not require paragraphs, you may take advantage of the `\abstracttext` macro (see the comment in this TEX file below).

Keywords

Aalto University
School of Science

Tekijä Antti-Jussi Asunmaa

Työn nimi Context is king - Retrieval-Augmented Generation in enterprise-ready systems

Koulutusohjelma Computer, Communication and Information Sciences

Pääaine Computer Science

Työn valvoja Prof. Petri Vuorimaa

Työn ohjaaja DI. Lauri Salminen

Yhteistyötaho Nordea Bank Oyj

Päivämäärä 30.6.2026 **Sivumäärä** 61+13 **Kieli** englanti

Tiivistelmä

Tiivistelmä on lyhyt kuvaus työn keskeisestä sisällöstä usein yhtenä kappaleena: mitä tutkittiin ja miten sekä mitkä olivat tärkeimmät tulokset. Suomenkielisen opinnäytteen tiivistelmä kirjoitetaan suomeksi ja englanniksi ja ruotsinkielisen vastaavasti ruotsiksi ja englanniksi. Suomen- tai ruotsinkielisten opiskelijoiden, joiden opinnäytteen kieli on englanti, tulee kirjoittaa tiivistelmänsä englanniksi ja koulusivistyskielellään. Muiden kuin koulusivistyskieleltään suomen- tai ruotsinkielisten tulee kirjoittaa tiivistelmänsä vain englanniksi. Opiskelija voi halutessaan lisätä opinnäytteeseensä toisen tai kolmannen tiivistelmän omalla äidinkielellään. Tämän opinnäytteen tiivistelmäteksti kirjoitetaan opinnäytteen luettavan osan lomakkeen lisäksi myös pdf-tiedoston metadataan. Kirjoita tähän metadataan kirjoitettavaa teksti. Metadatatekstissa ei saa olla erikoismerkkejä, rivinvaiho- tai kappaleenjakomerkkiä, joten näitä merkkeja ei saa käyttää tässä. Jos tiivistelmäsi ei sisällä erikoimerkkejä eikä kaipaa kappaleenjakoa, voit hyödynttää makroa abstracttext luodessasi lomakkeen tiivistelmää (katso kommentti tässä TeX-tiedostossa alla). Metadatatiivistelmatekstin on muuten oltava sama kuin lomakkeessa oleva teksti.

Avainsanat RAG

Preface

I want to thank Professor Petri Vuorimaa and my instructor Lauri Salminen for their guidance.

I also want to thank my partner for keeping me sane and alive.

Otaniemi, 30 June 2026

Antti-Jussi Asunmaa

AI usage

Large language models (LLMs) and generative artificial intelligence tools, specifically Google Gemini, were utilized in multiple phases of this thesis. First, they assisted in the ideation phase to refine the research scope. Second, they were used to identify relevant literature, keywords, and search formulations for the literature review. Third, AI tools were employed for grammatical correction and to improve the stylistic flow and vocabulary of the thesis; however, all draft text was originally authored by the author. Lastly, AI-based code assistants were used to support code development for the empirical evaluation.

The author accepts full responsibility for the factual accuracy of this thesis and for any errors or omissions, whether originating from the author's own input or from the systems utilized.

Contents

Abstract 3

Abstract (in Finnish) 4

Preface 5

AI usage 6

Contents 7

List of Tables 9

Glossary & Abbreviations 11

1 Introduction 13

2 Technical background 15
2.1 Large-Language Models . . . 15
2.1.1 Evolution and Adoption of Language Models . . . 15
2.1.2 Architectural and Operational Constraints . . . 15
2.2 Naive Retrieval-Augmented Generation . . . 16

3 Literature review 19
3.1 Cascading . . . 19
3.2 Small-to-Big retrieval . . . 20
3.3 Semantic Re-ranking . . . 21
3.4 Prompt Caching . . . 23

4 System Architecture and Experimental Methodology 25
4.1 Research Methodology . . . 25
4.2 System Implementation and Models . . . 26
4.3 Optimization Pipeline . . . 27
4.4 Benchmark Datasets . . . 29
4.5 Evaluation Setup and Metrics . . . 30
4.5.1 Latency Profiles and Computational Overhead . . . 30
4.5.2 Operational Cost Accounting . . . 31
4.5.3 Quality Metrics . . . 32
4.5.4 Pareto Frontier and Trade-off Equilibrium . . . 34
4.6 Threats to Validity . . . 35

5 Results 38
5.1 Latency Profiles and Computational Overhead . . . 38
5.1.1 Academic Multi-Hop Benchmark (MuSiQue) . . . 38
5.1.2 Enterprise Production Corpus . . . 39
5.2 Operational Expenditure and Token Dynamics . . . 41

5.2.1 Academic Multi-Hop Benchmark (MuSiQue) 41
5.2.2 Enterprise Production Corpus 41
5.3 Retrieval and Generation Quality 43
5.3.1 Academic Multi-Hop Benchmark (MuSiQue) 43
5.3.2 Enterprise Production Corpus 45
5.4 Trade-off Equilibrium and Pareto Frontier 47
5.4.1 Academic Multi-Hop Benchmark (MuSiQue) 47
5.4.2 Enterprise Production Corpus 48
5.5 Executive Performance Delta Synthesis 50

6 Discussion **53**
6.1 The Prompt Caching Paradox: Latency Inversion versus Tail-Latency Protection 53
6.2 Model Cascading and the Frontier Over-Refusal Paradox 54
6.3 Multi-Module Interactions: Synergy Bifurcation and Attention Anchoring 54
6.4 Corpus Topology, the Retrieval Noise Threshold, and Enterprise Deployment 54
6.5 Methodological Reflections and Limitations 54

7 Conclusion **55**

A Enterprise Knowledge Corpus Extraction Methodology **62**

B Operational Cost Catalog and Cloud Tariffs **64**

C Detailed Benchmark Performance Profiles and Component Breakdowns **65**
C.1 Component-Wise Latency Breakdowns 65
C.2 Operational Expenditure Component Decompositions 66
C.3 Multi-Dimensional Quality Metric Profiles 67

D Supplementary Benchmark Evaluation on Unconstrained Raw Enterprise Corpus **69**

List of Tables

1 Primary research questions 14
2 Full factorial experimental design matrix 26
3 Two-tailed hypothesis formulations mapped to the research questions (treatment pipelines vs. naive baseline). 27
4 Architectural specifications of optimization modules 28
5 End-to-end execution latency and Time-to-First-Token (TTFT) across 16 pipeline architectures on MuSiQue. Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline. 39
6 End-to-end execution latency and Time-to-First-Token (TTFT) across 16 pipeline architectures on Enterprise Closed. Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline. 40
7 Operational cost and token consumption breakdown across 16 pipeline architectures on MuSiQue. Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline. 42
8 Operational cost and token consumption breakdown across 16 pipeline architectures on Enterprise Closed. Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline. 43
9 Retrieval, generation, and multi-hop reasoning quality metrics across 16 pipeline architectures on the MuSiQue benchmark ($N = 100$). Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline. 44
10 Retrieval, generation, and out-of-domain rejection quality metrics across 16 pipeline architectures on the Enterprise Knowledge Corpus ($N = 100$). Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline. 46
11 Multi-objective Pareto efficiency and 3D non-domination trade-off metrics across 16 pipeline architectures on MuSiQue. 47
12 Multi-objective Pareto efficiency and 3D non-domination trade-off metrics across 16 pipeline architectures on Enterprise Closed. . . . 49
B1 Operational pricing catalog and cloud tariffs for generative, re-ranking, and embedding services (verified as of September 2026). Generative model tariffs reflect standard pay-as-you-go pricing for $\leq$ 200K input tokens, corresponding to all evaluated multi-hop and enterprise workload requests. 64
D1 End-to-end execution latency and Time-to-First-Token (TTFT) across 16 pipeline architectures on the unconstrained raw Enterprise corpus ($N = 100$). Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline. 69

D2 Operational cost and token consumption breakdown across 16 pipeline architectures on the unconstrained raw Enterprise corpus ($N = 100$). Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline. 71
D3 Retrieval, generation, and out-of-domain rejection quality metrics across 16 pipeline architectures on the unconstrained raw Enterprise Knowledge Corpus ($N = 100$). Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline. 72
D4 Multi-objective Pareto efficiency and 3D non-domination trade-off metrics across 16 pipeline architectures on the unconstrained raw Enterprise corpus ($N = 100$). 74

Glossary & Abbreviations

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BLEU	Bilingual Evaluation Understudy
DCC	Document Contextual Coherence
E2E	End-to-End
EM	Exact Match
FRR	False Refusal Rate
HNSW	Hierarchical Navigable Small World
ICC	Intrachunk Cohesion
KV	Key-Value
LitM	Lost in the Middle
LLM	Large Language Model
MC	Model Cascading
MRL	Matryoshka Representation Learning
NDCG	Normalized Discounted Cumulative Gain
NLP	Natural Language Processing
OOD	Out-of-Domain
PC	Prompt Caching
PLM	Pre-trained Language Model
QPD	Quality per Dollar
QPS	Quality per Second
RAG	Retrieval-Augmented Generation
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
S2B	Small-to-Big
SLA	Service Level Agreement
SR	Semantic Re-ranking
TTFT	Time-to-First-Token

Glossary

Chunking	The process of dividing a large source text into smaller, distinct segments (chunks) prior to vector database insertion.
Context Window	The maximum amount of text (measured in tokens) that a language model can process in a single prompt iteration.
Cosine Similarity	A mathematical metric used to measure the semantic similarity between two vectors by calculating the cosine of the angle between them.
Embedding	A mathematical vector representation of text that captures its semantic and contextual meaning in a high-dimensional space.
Hallucination	A phenomenon where a language model generates output that is fluent and plausible but factually incorrect or unsupported by the training data.
Inference	The operational phase where a fully trained language model processes a user prompt and generates an output response.
Non-parametric Knowledge	Data or information that resides outside the language model's internal weights, typically stored in an external database, and retrieved dynamically during inference.
Parametric Knowledge	The static, internal information that is permanently embedded within a language model's weights during its training phase.
Time-to-First-Token	A performance metric that measures the latency from the moment a prompt is sent to the model until it begins generating its very first output token.
Token	The foundational atomic unit of text (such as a word, character, or sub-word) into which input strings are split before being processed by a language model.
Vectorization	The algorithmic process of converting textual data into numerical vectors (embeddings) using a machine learning model.

1 Introduction

Modern enterprise-grade AI systems face continuously increasing resource constraints, in addition to growing operational expenditures driven by a rapid acceleration in corporate AI integration and exponential growth in the demand for AI computing capacity [1]. Although API providers frequently lower unit token pricing, overall costs continue to rise due to explosive token usage and the complexity of multi-step agentic workflows that use the latest foundational large language models (LLMs) [2, 3]. This highlights the critical need for optimization across all phases of enterprise AI systems to ensure that computing resources are utilized as efficiently as possible.

This is especially true for Retrieval-Augmented Generation (RAG), which serves as a primary solution to mitigate critical limitations of modern LLMs, such as hallucination, the lack of up-to-date data, and the absence of domain-specific knowledge [4, 5]. By design, RAG inherently increases token consumption, particularly input tokens, as the core concept relies on prepending retrieved document context to user queries to generate more grounded responses. Within the retrieval module, a central challenge lies in finding the optimal balance between retrieval efficiency and the quality of the retrieved information [5]. While various optimization strategies have been proposed in contemporary literature [5, 6], they are typically evaluated on static benchmarks and lack direct integration with industry-standard, production-grade languages and frameworks such as Java and Spring AI [7].

The primary objective of this thesis is to investigate state-of-the-art optimization strategies for RAG systems, and the main contribution is to evaluate how these strategies perform within enterprise environments. Furthermore, this work proposes a concrete architectural solution to implement these optimization strategies utilizing enterprise frameworks. To establish a focused scope, four specific optimization strategies are analyzed in detail: model cascading [2], small-to-big retrieval [8], semantic re-ranking [9], and prompt caching [10].

Each of these methods addresses a distinct phase of the RAG pipeline. Model cascading and small-to-big retrieval optimize the query and retrieval stages, whereas semantic re-ranking refines the retrieved results to minimize prompt size and maximize contextual relevance. Prompt caching, on the other hand, stores the precomputed key-value (KV) attention states of static prompt prefixes in memory, thereby accelerating the generator's prefill phase and reducing input token costs.

Many of these optimization strategies are typically evaluated in isolation, leaving a gap in the literature regarding how they interact when combined. For example, expanding context windows via small-to-big retrieval increases the proportion of dynamic, uncached tokens relative to the static cached prefix, potentially diluting the relative latency and cost savings of prompt caching. In addition, while model cascading may increase *per-query* latency for complex tasks routed to more capable models, it decreases *average* latency overall by resolving simpler tasks with faster, less capable models.

Evaluating and resolving these interactive conflicts is crucial for high-volume enterprise applications, such as customer support systems, where operational requirements are exceptionally rigorous. In these environments, minor delays and incremental token usage quickly compound into systemic instability, elevated latencies, and escalating operational costs. Furthermore, hallucinated or erroneous outputs present substantial risks of reputational damage and potential financial liabilities [11]. Because enterprise systems must strictly navigate these critical constraints, cost, latency, and output quality are established as the primary evaluation metrics for this thesis. To guide this investigation, this study addresses the four primary research questions summarized in Table 1.

Table 1: Primary research questions addressed in this thesis.

	Research Question
RQ1	**Latency:** How do small-to-big retrieval, semantic re-ranking, prompt caching, and model cascading affect the latency profile (specifically Time to First Token [TTFT] and overall end-to-end execution latency) of a RAG system?
RQ2	**Cost:** To what extent do these optimization methods reduce the operational cost of the system, accounting for token pricing tiers, prompt caching discounts, and model cascading?
RQ3	**Quality:** How do these optimizations impact retrieval and generation quality (measured by Context Precision, Context Recall, Faithfulness, and Answer Relevance) compared to a naive RAG baseline?
RQ4	**Interaction:** How do these optimization methods interact when combined, and what is the optimal equilibrium between latency, cost, and quality in an enterprise customer support system?

The rest of the thesis is structured as follows: Section 2 briefly explains what LLMs and RAG are, while Section 3 examines the current literature on RAG optimization methods. Research methods are introduced in Section 4, and the results are discussed in Section 5. Finally, the discussion is presented in Section 6, and concluding remarks are provided in Section 7.

2 Technical background

RAG offers an architectural solution to mitigate several of the most fundamental limitations of LLMs, such as hallucination, the lack of up-to-date data, and the lack of domain-specific knowledge [5]. To establish a comprehensive understanding of these concepts, this chapter provides the necessary technical background for both technologies. The first section examines the architectural foundations and inherent drawbacks of LLMs[1], while the subsequent section focuses on the core principles and operational mechanics of plain vanilla RAG.

2.1 Large-Language Models

2.1.1 Evolution and Adoption of Language Models

Natural language processing (NLP) has traditionally focused on enabling computer systems to understand human-generated language and acquire knowledge [13]. Early statistical approaches, such as n-gram language models, attempted to resolve the inherent ambiguity of natural language by predicting the probability distribution of word sequences [13]. While these traditional models could perform structured tasks like rule-based information extraction or spam filtering, their capability to comprehend semantic nuances and manage complex linguistic tasks remained limited [14].

Many pre-trained language models (PLMs), such as GPT-2 [15], are based on the transformer architecture [16], which excels at handling long-range dependencies through parallel training. While early PLMs required task-specific fine-tuning to perform downstream applications [17], modern LLMs scale this architecture to hundreds of billions of parameters. According to Zhao et al. [14], the transition from PLMs to LLMs is primarily characterized by this massive scale in parameters and training data, which gives rise to emergent abilities, enabling the models to solve complex tasks and follow instructions without explicit task-specific retraining.

Presently, LLMs are among the most widely used technologies behind modern generative AI applications [14]. Even though these models have demonstrated strong performance across various tasks, such as coding [18], and have been widely adopted[2], they are not perfect.

2.1.2 Architectural and Operational Constraints

Despite their capabilities, LLMs exhibit several major limitations, including biases, hallucinations, and the "Lost in the Middle" (LitM) phenomenon [19, 20]. Bias refers to the fact that the model's output heavily depends on its training data, meaning that the generated content may include inappropriate language, conspiracy theories, or gender bias [19].

[1] This section is partly based on the author's bachelor's thesis [12].

[2] For instance, data aggregated in the report [1] highlights a rapid acceleration in corporate AI integration. In Europe, corporate adoption in at least one business function was 91% in 2025 compared to 57% in 2023.

Due to the autoregressive nature of modern LLMs, these models are inherently prone to hallucinations, where they produce outputs that appear plausible but are factually incorrect [21]. This occurs because the models lack an intrinsic understanding of facts, but instead, they generate responses by predicting subsequent tokens based on statistical patterns learned during training. This relying mechanism draws entirely from the model's parametric knowledge, meaning the information is permanently embedded within the model's weights during the training phase [17]. This static nature, consequently, makes it computationally and environmentally expensive and impractical to frequently retrain or fine-tune entire models when the underlying information required to complete a task changes [1].

While the context windows, i.e., LLMs' input sizes, have grown significantly during the last couple of years, allowing models to process up to millions of input tokens [22, 23], these architectures consistently suffer from the LitM phenomenon. This problem refers to a limitation where models effectively utilize information located at the absolute beginning (primacy bias) or the end (recency bias) of the prompt, while failing to retrieve relevant facts buried within the middle sections [20]. In addition, utilizing massive contexts severely increases computational latency and operational expenses, as modern cloud LLM APIs employ token-based pricing models where input costs scale linearly with prompt length and often double when exceeding specific context thresholds [3]. These inherent limitations effectively prevent the full utilization of large context windows in production-grade enterprise environments when attempting to ingest massive enterprise knowledge bases directly into the prompt.

Another source of errors in the output is that the training data has a cutoff date, so it does not contain the most recent information and may generate incorrect answers to questions about recent events [24]. To mitigate these limitations, modern LLM-based systems are integrated with web search capabilities to retrieve real-time information from the internet, a paradigm used, for example, by the Google Gemini model family [22]. While this integration helps keep responses up to date, it introduces an external dependency on the reliability of internet sources. Furthermore, relying purely on open-web search does not solve the challenge for enterprise environments, where the required knowledge is locked behind internal intranet pages and private corporate databases rather than publicly accessible web pages.

2.2 Naive Retrieval-Augmented Generation

To address the limitations detailed in the previous section, Retrieval-Augmented Generation (RAG) offers an architectural solution that decouples the LLM's reasoning capabilities from its static parametrized memory [4]. RAG enriches the LLMs' parametric knowledge with non-parametric data served dynamically from an external knowledge base. The LLM's output is based on the user's initial prompt, augmented with the data retrieved from the external knowledge base, and therefore forces the LLM to ground its response in the provided context rather than its parametrized data.

The retrieval-augmentation process requires both the source knowledge and the

user query to be preprocessed. Preprocessing of the knowledge base contains three phases: chunking, vectorizing, and indexing. During this data preparation phase, source documents—such as PDF files—are chunked into smaller, more manageable segments[3]. These chunks are converted into numerical vectors using an embedding model so that their semantic meanings can be mathematically calculated, and they are subsequently stored in an indexed vector database [26]. During runtime inference, the user's initial prompt is vectorized using the same embedding model. The system then calculates the mathematical distance (e.g., cosine similarity) between the prompt vector and the stored vectors, retrieving a pre-defined number of the most relevant chunks—denoted as the top-k—from the database to augment the initial prompt [27].

To formalize this two-phase pipeline, the retrieval-augmentation process can be represented mathematically. In the first phase, a document corpus is split into N smaller text chunks, forming the external knowledge base $D = \{d_1, d_2, \ldots, d_N\}$. These chunks are then mapped into a vector database using an embedding function $E(\cdot)$, which yields the vector representation $\mathbf{v}_i = E(d_i)$ for each chunk $d_i \in D$.

In the second phase, the user's query q is embedded into a vector form using the same embedding function $E(q)$, after which the top k most relevant chunks R_q are fetched based on their cosine similarity score using the retrieval function Ret_k:

$$\text{Ret}_k(q, D) = R_q. \tag{1}$$

Finally, the query q and the retrieved chunks R_q are fed to the generator model $G(\cdot)$ to produce the final answer a, denoted as

$$G(q, R_q) = a. \tag{2}$$

The primary advantage of the RAG architecture lies in its reliance on an external knowledge base. When properly prompted, this mechanism significantly mitigates hallucinations by forcing the LLM to ground its output exclusively on the retrieved context [28]. Furthermore, maintaining an external data repository is substantially more cost-effective and computationally practical than continuously retraining or fine-tuning the underlying model [26], allowing enterprise systems to remain up-to-date in real time. From a data governance perspective, an external knowledge base ensures that sensitive corporate data remains secure and under internal control [29]. Lastly, by selectively retrieving only the most relevant documents, RAG compresses the required input context, thereby reducing API transaction costs and inference latency compared to naive long-context processing.

However, these benefits introduce distinct trade-offs. The major drawback of a naive RAG pipeline is the increased end-to-end latency of the system [26]. This latency overhead stems primarily from sequential network requests: first querying the embedding model and vector database, and subsequently transmitting the expanded

[3]Early architectures like Lewis et al. [4] utilized fixed-length segments of 100 words, while modern architectures optimize segment lengths [25].

prompt to the LLM. While modern vector distance calculations are highly optimized, the network overhead and the increased time-to-first-token (TTFT) caused by processing larger, augmented prompts add noticeable delay [26]. Consequently, if the RAG pipeline is not properly optimized, the final prompt can grow excessively large, rendering the production system both financially expensive and operationally slow.

3 Literature review

This chapter reviews the current literature on the four selected optimization techniques, focusing on the performance benefits each method provides in relation to the research questions. Relevant publications were identified on Google Scholar using keyword combinations of the optimization method and "RAG" or "retrieval-augmented generation". In addition, snowballing [30], starting from comprehensive surveys (e.g., [31, 26, 5]), was used to discover further relevant studies. Due to the rapid pace of development in this research area, the selected literature consists primarily of articles published after 2023. The remainder of this chapter is structured as follows: Section 3.1 reviews model cascading and routing, Section 3.2 examines small-to-big retrieval, Section 3.3 covers semantic re-ranking, and Section 3.4 discusses prompt caching.

3.1 Cascading

Modern RAG systems encounter diverse workloads, ranging from simple informational queries to highly complex analytical tasks. In production-grade enterprise environments—where high availability, low latency, and cost efficiency are critical constraints—a "one-size-fits-all" architecture is fundamentally sub-optimal [32, 2]. It over-allocates expensive computational resources for trivial questions, yet may lack the retrieval depth and reasoning capacity required for complex queries. To mitigate this, model cascading or query routing can be employed to dynamically direct inputs to an appropriately scaled configuration of the RAG pipeline [33]. For instance, a simple query can bypass the full pipeline and utilize a lightweight LLM with minimal context. Conversely, a complex query requiring synthesis across multiple enterprise knowledge bases can trigger an augmented pipeline with extended retrieval top-k parameters and a more capable foundation model.

To optimize these trade-offs systematically, these dynamic architectures can be formalized mathematically. In a classification-based query routing system, the selection process is modeled by defining a set of available generator models, $\mathcal{G} = \{G_1, G_2, \ldots, G_m\}$, ordered by their capacity and cost. Based on the estimated complexity of the input query q, a routing classifier

$$f_{\text{route}}(q) = j \in \{1, \ldots, m\} \tag{3}$$

maps it to a specific model index. Consequently, the query q and the retrieved chunks R_q are fed to the selected generator model $G_j(\cdot)$ to produce the final answer a:

$$G_j(q, R_q) = a. \tag{4}$$

Model cascading offers substantial operational cost reductions both in isolated LLM invocations and in RAG pipelines whose overall cost profiles are dominated by API usage. For instance, FrugalGPT [2] routes individual requests sequentially from lightweight to heavyweight models based on a reliability score generated by a scoring classifier. This cascading approach achieves up to a 98.3% cost reduction compared

to a homogeneous GPT-4 baseline while maintaining comparable response quality. Similarly, RouteLLM [33] yields a 1.14× to 3.66× cost reduction compared to a static GPT-4 deployment by dynamically bypassing expensive models, preserving approximately 95% of the baseline's quality. Similar cost reduction patterns have also been demonstrated by other adaptive RAG frameworks [32, 34].

Routing low-complexity queries to smaller models yields significant latency benefits due to their faster inference speeds. Crucially, the computational overhead introduced by the routing decision itself remains minimal, typically amounting to approximately 11 ms per query [35] with an added routing cost overhead below 0.4% [33]. In straightforward scenarios, bypassing retrieval operations entirely can yield order-of-magnitude improvements: Jeong et al. [32] reports a reduction in generation latency from 3.08 and 27.18 seconds (for single-step and multi-step retrieval pipelines) to 0.35 seconds. Furthermore, adaptive stopping mechanisms, such as that proposed by Park et al. [36], can prune unnecessary retrieval-generation loops, reducing the total number of iterations by 14–49%.

While providing both operational cost and latency benefits, query routing has a modest impact on quality. For example, Jeong et al. [32] found that the F1 score remained practically unchanged, while achieving a substantial reduction in latency (∼56%) and in token consumption (63%) compared to a multi-step retrieval baseline (using GPT-3.5). Similar results have been shown by Guo et al. [34], where token consumption decreased by up to ∼90% while losing only 0.8 points on F1, but improving the Exact Match score by 2.6 points compared to the agentic baseline, and yielding 20.7 and 15.5 percentage point improvements in F1 and Exact Match scores, respectively, compared to naive RAG.

3.2 Small-to-Big retrieval

Document chunking is a foundational phase in RAG pipelines, directly determining the granularity of the vector index. However, selecting a static chunk size introduces a fundamental trade-off between retrieval precision and contextual completeness. While smaller chunks are computationally efficient and exhibit high intrachunk cohesion (ICC), they frequently suffer from low document contextual coherence (DCC) due to semantic fragmentation [25]. Conversely, larger chunks preserve rich contextual relationships but degrade retrieval accuracy by diluting query-relevant semantic features within dense vector representations. Small-to-big retrieval resolves this tension by decoupling the representation used for vector search from the context provided to the generator. In this architecture, dense retrieval is executed over fine-grained index nodes, while the generation phase utilizes a broader, mapped context (such as parent chunks or entire source documents) retrieved via sibling pointer mapping. Implementations of this technique range from neighboring chunk expansion, which dynamically retrieves adjacent sliding windows to restore structural continuity [8], to hierarchical abstract trees, which map granular leaf nodes to high-level thematic summaries [37].

To abstract these various approaches, the underlying retrieval process is formally

modeled by representing the search space as a set of fine-grained index nodes $D_{\text{small}} = \{d_1, \ldots, d_N\}$ and a context-mapping function $g : D_{\text{small}} \to D_{\text{parent}}$ that maps each small chunk to its corresponding parent chunk or expanded context window. The retriever first operates on the small chunks:

$$\text{Ret}_k(q, D_{\text{small}}) = R_q^{\text{small}}. \tag{5}$$

Finally, the query q and the mapped parent contexts are fed to the generator model $G(\cdot)$ to produce the final answer a:

$$G\left(q, \{g(d) \mid d \in R_q^{\text{small}}\}\right) = a. \tag{6}$$

Different chunking and representation methods directly increase the accuracy of RAG pipelines. For example, Lumer et al. [8] showed that expanding the context to neighboring chunks can increase the win rate to 65% compared to using single independent chunks. In addition, the adaptive chunking method, where chunks are generated dynamically based on document structure, increases downstream answer correctness by 8–10 percentage points without requiring downstream prompt modifications [25]. Hierarchical abstract trees like Bridge-RAG [37] yield up to a 44% improvement in BLEU scores on complex reasoning tasks by tracing parent-child lineages. Similarly, dynamic reasoning frameworks like LogicRAG [38] and QPaug [39] increased string accuracy by 14.7% absolute (compared to the next best graph baseline) and answer F1 scores by over 34% relatively.

While small-to-big strategies provide substantial benefits to answer correctness and retrieval recall, they introduce operational and computational overhead. At query time, asynchronous parallel search for neighboring chunks [8] keeps latency minimal, adding a negligible ≈ 0.17 s overhead. However, more complex reasoning structures like LogicRAG [38] more than double the query latency compared to naive flat retrieval, increasing it from 4.28 s to 9.83 s. Furthermore, advanced chunking and indexing methods are highly time-consuming during the ingestion phase. The slowest model-driven and layout-aware chunking methods can run for tens of hours, whereas fixed-size chunking completes in under a second [40]. In hierarchical structures, generating node summaries and index trees also increases offline preprocessing costs (e.g., scaling from \$1.19 to \$6.12 per average filing in financial domains) [8]. Nonetheless, this preprocessing latency is typically acceptable in enterprise environments, as chunking and indexing are offline processes that occur only during initial document ingestion.

3.3 Semantic Re-ranking

In RAG pipelines, retrieved chunks might exhibit high mathematical similarity (e.g., cosine similarity) but lack actual semantic relevance to the query. The worst-case result of this semantic discrepancy is incorrect generation, compounded by the unnecessary token overhead of feeding irrelevant data to the generator. Semantic re-ranking is an optimization method that employs a multi-stage retrieval architecture to address

this issue. First, a fast and computationally efficient bi-encoder (such as dense vector embeddings or sparse BM25 retrieval) is used to retrieve a large candidate set (e.g., top-100) to optimize the recall phase. Subsequently, a computationally intensive cross-encoder reranking model evaluates deep query-document interactions to refine the results and select the top-k most relevant chunks before the generation of the final answer [41, 42]. Another key benefit of semantic re-ranking is its ability to mitigate the LitM phenomenon [20] by sorting the most semantically relevant chunks to the top of the context window. Furthermore, by filtering out irrelevant chunks before the generation stage, the reranker compresses the final prompt size, which directly reduces both input token consumption and TTFT.

To model this process systematically, semantic re-ranking is formalized as a two-stage retrieval pipeline. Let $D = \{d_1, d_2, \ldots, d_N\}$ represent the database of document chunks, and let q denote the user query. In the first stage, a computationally efficient bi-encoder retriever selects a large candidate set $C_1 \subset D$ of size K_1 (where $K_1 \gg k$):

$$R_{K_1}(q, D) = C_1. \tag{7}$$

In the second stage, a cross-encoder model reranks the candidates in C_1 based on deep query-document interactions to select the top-k most relevant chunks, forming the pruned context $C_2 \subset C_1$:

$$\text{Rerank}_k(q, C_1) = C_2. \tag{8}$$

Finally, the query q and the pruned context C_2 are fed to the generator model $G(\cdot)$ to produce the final answer a:

$$G(q, C_2) = a. \tag{9}$$

Semantic re-ranking has been shown to significantly improve both the retrieval and generation quality of RAG pipelines. For instance, in open-domain question-answering, Moreira et al. [41] demonstrated that a second-stage neural reranker yields a 14% relative and a 9 percentage point absolute improvement in average NDCG@10 across the BEIR QA datasets compared to standard bi-encoder retrievers. This improvement translates directly to downstream accuracy: by filtering out non-contributing documents, Wang et al. [43] increased Exact Match (EM) scores by 17.8 percentage points on NaturalQA and 17.4 percentage points on PopQA compared to naive RAG, while outperforming alternative self-reflective and corrective frameworks on TriviaQA with a 76.2% EM score. Furthermore, rather than relying on separate pipeline components, Yu et al. [9] consolidated ranking and generation into a single multi-task model, achieving 50.6% EM on NaturalQA, 82.9% EM on TriviaQA, and 57.6% EM on PopQA—outperforming standard chat-adapted baselines by 5.0 to 8.2 percentage points. When models are optimized for explicit denoising using self-synthesized rationales, they achieve relative accuracy improvements of 8.3% and demonstrate robust performance as the candidate count scales [44]. Finally, in domain-specific applications where precision is safety-critical, hybrid retrieval paired with a cross-encoder reranker increases high-correctness answers from 33.5% to 49.0% while reducing severe hallucinations and completely incorrect outputs from 35.3% to 22.5% [42].

Increased quality in semantic re-ranking comes with the explicit cost of increased latency while providing implicit potential for token savings. The key overhead arises during the second-stage retrieval, where the computationally intensive reranking model remains the primary bottleneck. For example, Moreira et al. [41] reports that using a pruned 4-billion-parameter reranker adds approximately 266 ms of latency on average. Similarly, incorporating the ranking mechanism directly into the generator model, as done in RankRAG, introduces a 0.9× to 6.0× latency overhead when evaluating between 20 and 100 candidate documents [9]. While downstream token savings are rarely quantified empirically in these studies, overall token consumption is implicitly reduced either by dynamically filtering out irrelevant documents below a relevance threshold [43] or by scaling down the final context window size since the most relevant documents are prioritized at the top [9].

3.4 Prompt Caching

RAG pipelines incur substantial TTFT latency and computational overhead because queries are continuously augmented with extensive retrieved context. However, much of this content is static or repetitive—such as system instructions, templates, and recurring document chunks. Prompt caching avoids redundant prefill passes by persisting intermediate key-value (KV) attention states in memory for reuse across requests. Modern architectures primarily use two paradigms: model-level prefix KV caching and application-level semantic completion caching. Prefix KV caching stores attention tensors for static prefixes or document blocks to bypass redundant prefill computation. Conversely, semantic completion caching stores historical query–response pairs, directly returning pre-computed answers when a query exceeds a semantic similarity threshold.

To formalize context-level KV caching systematically, the constructed prompt $P(q, R_q)$ is partitioned into a static prefix P_{static} and a dynamic, query-specific suffix $P_{\text{dynamic}}(q)$:

$$P(q, R_q) = [P_{\text{static}}, P_{\text{dynamic}}(q)]. \tag{10}$$

Once P_{static} has been pre-computed, its key-value attention states $KV(P_{\text{static}})$ are retrieved directly from the cache, allowing the generator model $G(\cdot)$ to evaluate self-attention exclusively over the dynamic suffix and produce the final answer:

$$G(P_{\text{dynamic}}(q) \mid KV(P_{\text{static}})) = a. \tag{11}$$

Prompt caching yields substantial latency reductions and operational cost savings across diverse RAG architectures. From a commercial deployment perspective, major cloud API providers offer prompt caching discounts up to a 10× cost reduction for cached prompt tokens [3]. At the infrastructure level, Gim et al. [10] reported 5×–10× TTFT speedups on VRAM-cached GPUs (1.5×–3× on host DRAM)—slashing latency from 900 ms to 90 ms on Llama-7B with 3K tokens—and up to 70× on CPUs. Similarly, Jin et al. [45] achieved a 4.0× TTFT reduction over vLLM+Faiss alongside a 2.1× throughput speedup via multi-tier knowledge tree caching. Broader evaluations

in TurboRAG [46], CacheBlend [47], and FusionRAG [48] confirm speedups ranging from 2.2× (CacheBlend) to 9.39× (FusionRAG). Although linear memory scaling presents a key trade-off (e.g., 2.5 GB RAM per 1K tokens for Llama-70B), tiered architectures mitigate VRAM constraints by offloading inactive KV blocks to DRAM or NVMe SSDs. Furthermore, by recomputing only critical tokens ($\leq$ 15% in FusionRAG and 15% in CacheBlend), these frameworks eliminate up to 82.3% of redundant prefill computation while sustaining cache hit rates as high as 77.9%.

Beyond latency and cost reductions, prompt caching maintains generation quality comparable to full prefill passes, with architectures like RAGCache [45] demonstrating zero quality loss across standard benchmarks. The primary challenge of prompt caching lies in uncaptured cross-chunk semantics and positional displacements, where identical context blocks exhibit varying semantics or positional shifts across queries, leading to cache misses and potential accuracy degradation. To overcome these limitations, recent approaches introduce selective recomputation strategies. For example, CacheBlend [47] selectively recomputes cross-attention for the top 15% most sensitive tokens (from a candidate set of 10–20% high-deviation tokens), restoring quality to within 0.01–0.03 absolute F1/ROUGE-L score of full prefill. Similarly, FusionRAG [48] pairs offline similarity-guided preprocessing with online query-guided attention selection—recomputing under 15% of tokens—to recover approximately 99% of the full attention quality.

4 System Architecture and Experimental Methodology

This chapter establishes the experimental framework and software architecture used to empirically evaluate the performance, operational cost, and output quality of RAG optimization techniques. To address the research questions formulated in Section 1, this study implements a quantitative benchmarking suite built upon an enterprise-ready Java and Spring AI infrastructure connected to Google Cloud Platform (GCP) Vertex AI services. The remainder of this chapter is structured as follows: Section 4.1 details the overall research approach and experimental design, while Section 4.2 describes the software architecture. Section 4.3 outlines the specific optimization strategies evaluated. Section 4.4 introduces the target benchmark datasets, Section 4.5 specifies the quantitative evaluation metrics and measurement instrumentation, and Section 4.6 analyzes potential threats to experimental validity.

4.1 Research Methodology

The empirical research is conducted using quantitative benchmarking following a two-stage experimental strategy. In the first stage, each optimization module is evaluated in isolation against the naive RAG baseline (C_0) to quantify its individual performance characteristics. In the second stage, combinatorial configurations—spanning pairwise pairings, multi-module triplets, and the fully integrated full-stack pipeline—are systematically tested against the baseline to identify the Pareto frontier and investigate systemic trade-offs between combined techniques. The complete experimental design matrix is presented in Table 2. This evaluation framework directly addresses the research questions formulated in Table 1. To ensure statistical reliability, two unmeasured warm-up queries precede the evaluation of the benchmark suite. Each configuration in the $2^4 = 16$ experimental matrix is evaluated across the 100 benchmark queries per dataset (yielding a sample size of $N = 100$ paired observations per dataset, or $N = 200$ across both corpora), resulting in 3,200 total measured pipeline invocations across the experimental matrix.

Due to the non-deterministic characteristics of distributed cloud systems and network environments, the benchmarked latency and cost metrics are expected to exhibit non-normal distributions. Consequently, the non-parametric Wilcoxon signed-rank test is employed to evaluate the statistical significance of observed differences between each optimization configuration ($C_1, \ldots, C_{15}$) and the naive baseline (C_0). Because each pipeline configuration is evaluated across an identical, paired set of benchmark queries, the paired signed-rank formulation is strictly applicable.

Tests are conducted independently across each evaluation metric using a two-tailed formulation to detect both performance enhancements and potential degradation without directional bias. To mitigate the inflated risk of Type I errors resulting from multiple pairwise comparisons (15 configurations across multiple metrics), family-wise error rates are controlled by applying the Holm-Bonferroni correction to all raw

Table 2: Full factorial experimental design matrix ($2^4 = 16$ configurations) across the two-stage benchmarking framework.

Config	Pipeline Composition	M_1: S2B	M_2: SR	M_3: PC	M_4: MC
Baseline ($k = 0$)					
C_0	Naive RAG Baseline	–	–	–	–
Stage 1: Single-Factor Isolated Testing ($k = 1$)					
C_1	S2B	✓	–	–	–
C_2	SR	–	✓	–	–
C_3	PC	–	–	✓	–
C_4	MC	–	–	–	✓
Stage 2: Pairwise Combinations ($k = 2$)					
C_5	S2B + SR	✓	✓	–	–
C_6	S2B + PC	✓	–	✓	–
C_7	S2B + MC	✓	–	–	✓
C_8	SR + PC	–	✓	✓	–
C_9	SR + MC	–	✓	–	✓
C_{10}	PC + MC	–	–	✓	✓
Stage 2: Multi-Module Triplets ($k = 3$)					
C_{11}	S2B + SR + PC	✓	✓	✓	–
C_{12}	S2B + SR + MC	✓	✓	–	✓
C_{13}	S2B + PC + MC	✓	–	✓	✓
C_{14}	SR + PC + MC	–	✓	✓	✓
Stage 2: 4-Module Integration ($k = 4$)					
C_{15}	S2B + SR + PC + MC	✓	✓	✓	✓

Notes: M_1: Small-to-Big Retrieval (S2B); M_2: Semantic Re-ranking (SR); M_3: Prompt Caching (PC); M_4: Model Cascading (MC). Checkmarks (✓) denote active optimization components in the pipeline.

p-values. Statistical significance is evaluated against a threshold of $\alpha = 0.05$. Formal hypothesis definitions mapped to the research questions are presented in Table 3. Note that RQ4 examines multi-objective trade-offs and is evaluated using Pareto frontier analysis (Section 4.5.4) rather than a statistical hypothesis test.

4.2 System Implementation and Models

The experimental testbed is built upon a production-grade enterprise software stack. The application is implemented in Java 21 LTS using Spring Boot 4.1.0 and Spring AI version 2.0.0. Relational metadata and vector embeddings are managed in PostgreSQL version 16 with the `pgvector` extension (version 0.8.6), utilizing Hierarchical Navigable Small World (HNSW) indexing with cosine distance [4]. Both the benchmarking suite and the PostgreSQL database are executed locally on a laptop workstation equipped with an Intel Core Ultra 7 processor and 32 GB of RAM. All foundation

Table 3: Two-tailed hypothesis formulations mapped to the research questions (treatment pipelines vs. naive baseline).

Target	Metric	Null Hypothesis (H_0)	Alternative Hypothesis (H_1)
RQ1	Latency (TTFT, E2E)	$\tilde{\mu}_{\Delta\text{Latency}} = 0$	$\tilde{\mu}_{\Delta\text{Latency}} \neq 0$
RQ2	Cost (USD / tokens)	$\tilde{\mu}_{\Delta\text{Cost}} = 0$	$\tilde{\mu}_{\Delta\text{Cost}} \neq 0$
RQ3	Quality (RAGAS)	$\tilde{\mu}_{\Delta\text{Quality}} = 0$	$\tilde{\mu}_{\Delta\text{Quality}} \neq 0$

Notes: $\tilde{\mu}_{\Delta}$ represents the median paired difference between a treatment pipeline and the naive baseline. Tests are evaluated at significance level $\alpha = 0.05$.

model invocations are transmitted via HTTPS to GCP Vertex AI, with generative models routed through GCP global endpoints.

The primary foundation model for generation is Google Gemini 3.5 Flash; for model cascading (M_4), Gemini 3.5 Flash-Lite and Gemini 3.1 Pro are utilized as low-latency and high-capacity tiers, respectively. Dense vector representations are generated using `gemini-embedding-001`, configured with an output dimensionality of 768 via Matryoshka Representation Learning (MRL) [49] to balance retrieval efficiency with storage requirements. For two-stage semantic re-ranking (M_2), the GCP Vertex AI Ranking API with the `semantic-ranker-default-004` cross-encoder model is employed.

4.3 Optimization Pipeline

Each optimization method encompasses a broad architectural design space. For example, chunk token limits and overlap ratios vary substantially across RAG literature. Furthermore, differences in foundation model parameter scales, routing thresholds, and caching configurations significantly expand the potential parameter space. To ensure rigorous and reproducible evaluation, the technical implementation and baseline configurations evaluated in this study are summarized in Table 4. The selected parameters balance empirical prevalence in the literature review (Section 3) with practical feasibility in enterprise software stacks.

Naive Baseline Configuration (C_0). Due to the structure of the evaluation datasets, a maximum chunk size of 512 tokens is adopted without sliding window overlap. Specifically, the MuSiQue corpus naturally consists of pre-segmented Wikipedia paragraphs (typically 100–150 tokens) that are ingested without additional splitting, whereas the Enterprise corpus is partitioned into logical document sections capped at 512 tokens. In addition, dense vector retrieval of top-$k = 5$ is chosen as the baseline configuration, reflecting standard experimental benchmarks in recent literature [40, 4].

Table 4: Architectural specifications and technical implementation of the evaluated RAG optimization modules.

Module	Optimization Strategy	Variables and Configuration
C_0	Naive RAG (Baseline)	Chunk size: ≤ 512 tokens, retrieval: top-$k = 5$.
M_1	Small-to-Big Retrieval	Child chunk: ≤ 128 tokens, parent window: ≤ 1024 tokens, retrieval: top-$k = 5$.
M_2	Two-Stage Semantic Re-ranking	Candidate pool: top-50, reranked context: top-$k = 5$, model: `semantic-ranker-default-004`.
M_3	Prompt Caching	Context threshold: $\geq 4\,096$ tokens, caching mechanism: GCP Vertex AI Explicit Context Cache.
M_4	Hybrid Model Routing	Routing targets: Router/1-hop $\rightarrow$ Gemini 3.5 Flash-Lite; 2-hop $\rightarrow$ Gemini 3.5 Flash; 3/4-hop $\rightarrow$ Gemini 3.1 Pro.

Small-to-Big Retrieval (M_1). Child chunks of 128 tokens expanding into 1024-token parent context windows are selected based on empirical configurations in recent hierarchical RAG studies [25, 37, 8]. Furthermore, when multiple retrieved child chunks originate from the same parent document, parent-ID deduplication is applied to retain the highest-ranked occurrence, preventing redundant token consumption and context duplication.

Two-Stage Semantic Re-ranking (M_2). The candidate retrieval pool for M_2 is generated using dense vector embeddings from `gemini-embedding-001` and re-ranked using the GCP Vertex AI Ranking API with the `semantic-ranker-default-004` cross-encoder model. The candidate pool size depends on the active pipeline composition: in isolated configurations (C_2), first-stage vector search retrieves a candidate pool of 50 baseline chunks to match the C_0 baseline granularity. In combined configurations ($C_5, C_{11}, C_{12}, C_{15}$), the first-stage vector search retrieves 50 child chunks of 128 tokens each, which are re-ranked directly by `semantic-ranker-default-004` before the top-5 winning chunks are expanded to their 1024-token parent context windows.

Prompt Caching (M_3). A minimum threshold of 4 096 tokens is selected to satisfy the operational boundary of Google Cloud Vertex AI Context Caching for static system instructions and pre-indexed knowledge corpora. To exceed this operational threshold naturally without synthetic padding, a static system prompt incorporating an in-context learning (ICL) library of eight extensive few-shot reasoning demonstrations ($\approx$ 7,150 tokens) is prepended to the context. Conversely, dynamically retrieved top-k document chunks and incoming user queries are decoupled from the cached prefix and processed

as dynamic input tokens during generation. To eliminate cold-start latency associated with initial cache allocation, two unmeasured pre-warming invocations precede each caching-enabled experimental run. Furthermore, implicit caching is disabled across all configurations, and active caches are explicitly cleared between distinct experimental runs to prevent cross-configuration state leakage.

Hybrid Model Routing (M_4). To replicate real-world enterprise deployments where query complexity is unknown a priori, an automated zero-shot classifier powered by Gemini 3.5 Flash-Lite dynamically evaluates incoming queries. Straightforward single-hop requests are handled directly by Gemini 3.5 Flash-Lite, intermediate queries requiring multi-passage synthesis are escalated to Gemini 3.5 Flash, and complex multi-hop reasoning tasks are routed to Gemini 3.1 Pro. Crucially, routing governs exclusively the generative model tier, while the retrieval pipeline, candidate context pool (top-$k = 5$), and prompt structure remain invariant to preserve factorial independence. When evaluated in conjunction with Prompt Caching (M_3), a dedicated pre-warmed context cache is maintained for each generative model tier prior to experimental execution. The latency overhead (T_{route}) and token costs of the classifier are explicitly recorded in the evaluation.

4.4 Benchmark Datasets

The primary academic benchmark dataset is MuSiQue [50], from which a sample of 100 question–answer pairs is selected following established benchmarking methodologies in recent RAG literature [8, 25]. Specifically, the sample comprises 40% 2-hop questions, 40% 3- and 4-hop, as well as 20% of non-answerable questions to ensure balanced coverage across varying reasoning difficulty tiers. For non-answerable queries, the expected reference response is a standardized refusal string (*"The provided context does not contain sufficient information to answer this query."*), allowing rejection accuracy and hallucination resistance to be quantitatively evaluated. Multi-hop queries eliminate the reasoning shortcuts and surface-level lexical matching common in single-hop datasets [50], making them well-suited for evaluating Small-to-Big retrieval (M_1), Semantic Re-ranking (M_2), and Prompt Caching (M_3). Furthermore, stratifying the evaluation set by reasoning depth provides a realistic testbed for assessing hybrid Model Cascading (M_4) across distinct query complexity tiers.

To complement the academic benchmark with domain-specific enterprise workloads, a secondary evaluation dataset comprising 100 synthetically generated question–answer pairs is constructed to reflect real-world customer inquiries regarding the products and services of the partner enterprise. The underlying knowledge corpus is compiled by scraping the public-facing documentation and web portal of the partner organization, with the comprehensive extraction methodology detailed in Appendix A. To evaluate pipeline robustness across varied interaction patterns, the dataset is partitioned into 40% single-hop factual queries, 40% multi-hop relational questions, and 20% out-of-domain queries. Because the enterprise corpus and customer inquiries are natively in Finnish, out-of-domain queries require the localized canonical refusal string (*"Annettu*

konteksti ei sisällä riittävästi tietoa kysymykseen vastaamiseksi."), while the canonical English string is retained as an accepted semantic alias during automated evaluation.

The enterprise dataset is evaluated in two distinct configurations. The main results presented in Chapter 5 are based on the closed corpus, which is refactored to match the structure and candidate density of the MuSiQue [50] benchmark to ensure direct comparability. Compared to the full crawled web data, this refactoring reduces the retrieval corpus size and eliminates the search space disparity as a confounding variable. The refactoring methodology is described in detail in Appendix A, while the pipeline's performance in the unconstrained, noisy raw corpus is evaluated in Appendix D.

4.5 Evaluation Setup and Metrics

The empirical evaluation of system performance, operational expenditure, and response quality is conducted using the industry-standard RAGAS evaluation suite [51] combined with enterprise-grade application telemetry. Within the RAGAS framework, the open-weights language model Qwen3.8-27B [52] is deployed locally via vLLM as an automated LLM-as-a-judge at zero temperature ($T = 0.0$) with greedy decoding to ensure deterministic evaluation scoring, while dense semantic similarity for Answer Relevancy is computed locally using the BGE-M3 embedding model [53]. The following subsections formally establish the measurement instrumentation and mathematical formulations for each research question.

4.5.1 Latency Profiles and Computational Overhead

The latency characteristics of the RAG pipelines are evaluated across two primary operational dimensions: Time to First Token (TTFT, in milliseconds) and overall End-to-End latency (T_{total}, in milliseconds). TTFT measures the duration elapsed from query submission until the generation model emits its initial response token, serving as a critical indicator of perceived user responsiveness in interactive systems. End-to-end latency quantifies the total wall-clock execution time required for the entire pipeline to complete generation and stream the response. To isolate the computational overhead introduced by individual architectural optimizations, fine-grained telemetry records the execution latency of each micro-phase:

$$\begin{aligned} T_{\text{total}}(q) = {} & T_{\text{embed}}(q) + T_{\text{route}}(q) + T_{\text{ret}}(q) + T_{\text{rerank}}(q) \\ & + T_{\text{expand}}(q) + T_{\text{cache}}(q) + T_{\text{prefill}}(q) + T_{\text{stream}}(q) \end{aligned} \tag{12}$$

where T_{embed} denotes query vectorization duration, T_{route} is the model cascading routing overhead, T_{ret} is the first-stage HNSW index traversal latency (with application telemetry recording and reporting their combined retrieval duration as $T_{\text{retrieval}} = T_{\text{embed}} + T_{\text{ret}}$ across all summary tables and figures), T_{rerank} is the cross-encoder semantic re-ranking duration, T_{expand} is the parent-document metadata resolution time, T_{cache} is the context

cache lookup and validation latency, T_{prefill} is the generator prefill duration, and T_{stream} is the streaming token generation duration. For pipeline configurations where specific optimization modules are inactive, their corresponding latency terms evaluate to zero. Under this formulation, user-perceived Time to First Token is defined as:

$$\text{TTFT}(q) = T_{\text{total}}(q) - T_{\text{stream}}(q) \tag{13}$$

Both median (P_{50}) and 95th percentile tail (P_{95}) latencies are reported across the steady-state executions to evaluate enterprise SLA compliance.

4.5.2 Operational Cost Accounting

Operational expenditure constitutes a critical bottleneck for enterprise RAG adoption at scale. To establish an accurate, reproducible economic model, execution costs are computed per query based on published Google Cloud Gemini Enterprise Agent Platform pricing tiers. The operational cost model incorporates dynamic model tier routing (M_4), context caching discounts (M_3), and auxiliary service invocation overhead (M_2). The total monetary cost $\text{Cost}(q)$ for processing query q is formally defined as:

$$\begin{aligned}\text{Cost}(q) = {} & N_{\text{embed}}(q) \cdot P_{\text{embed}} + N_{\text{uncached}}(q) \cdot P_{\text{prompt}}^{(m(q))} + N_{\text{cached}}(q) \cdot P_{\text{cached}}^{(m(q))} \\ & + N_{\text{comp}}(q) \cdot P_{\text{comp}}^{(m(q))} + N_{\text{rerank}}(q) \cdot P_{\text{rerank}} + N_{\text{route}}(q) \cdot P_{\text{route}}\end{aligned} \tag{14}$$

where $m(q) \in \{\text{Flash-Lite}, \text{Flash}, \text{Pro}\}$ represents the active generative model tier selected for query q, N_{embed} is the number of query tokens vectorized at unit rate P_{embed}, and N_{uncached} and N_{cached} represent uncached and cached prompt tokens billed at tier-specific rates $P_{\text{prompt}}^{(m(q))}$ and $P_{\text{cached}}^{(m(q))}$, reflecting model-specific caching discount rates detailed in Table B1. In Module M_3, the static prefix tokens are billed at the discounted rate $P_{\text{cached}}^{(m(q))}$, whereas dynamically retrieved chunks and runtime query tokens are accounted for under standard prompt pricing $P_{\text{prompt}}^{(m(q))}$. Furthermore, N_{comp} denotes the generated completion tokens billed at rate $P_{\text{comp}}^{(m(q))}$, N_{rerank} represents the count of candidate passages evaluated by the cross-encoder at per-document rate P_{rerank}, and $N_{\text{route}}(q) \cdot P_{\text{route}}$ accounts for the classification cost during model routing (M_4), covering both input and output token pricing of the router model. Fixed persistent infrastructure expenses, including time-based context cache storage fees, are invariant across configurations and negligible over the evaluation duration, and are therefore excluded from per-query operational cost calculations. The complete unit tariff schedule, encompassing all evaluated generative model tiers, cross-encoder ranking services, and embedding models alongside official vendor documentation references and acquisition timestamps (September 2026), is detailed in Appendix B (Table B1).

Because individual execution expenditures in highly optimized or cached architectures are fractional (frequently falling below $0.001 per request), presenting single-query costs directly introduces significant rounding truncation in empirical summaries. Consequently, operational expenditures in benchmark summary tables and Pareto frontier analyses are normalized and reported per 1,000 queries:

$$\text{Cost}_{1\text{k}} = 1000 \times \frac{1}{|Q|} \sum_{q \in Q} \text{Cost}(q) \tag{15}$$

providing an intuitive, standard enterprise pricing metric that transparently reflects macroscopic operational volume while preserving underlying per-query accounting fidelity.

4.5.3 Quality Metrics

Latency reduction and operational cost minimization are viable only if the response quality of the RAG pipeline satisfies end-user requirements. RAG pipelines exhibit two primary failure modes: retrieval failure and generation failure. Retrieval failure is defined as the inability to retrieve relevant context passages or the inclusion of irrelevant noise, whereas generation failure occurs when, despite retrieving relevant context, the generator produces an inaccurate response or hallucinated claims.

To evaluate quality across both stages of the pipeline, the RAGAS [51] framework provides dedicated metrics, formulated in Equations 16–19. While RAGAS is subject to the stochastic nature of LLM evaluation judges (discussed in Section 4.6), objective, deterministic token-level lexical overlap metrics—specifically token F_1 and Exact Match (EM)—are additionally computed across the complete query set ($N = 100$) against gold standard reference answers (including canonical refusal strings for unanswerable queries). Finally, a composite quality score is calculated to synthesize overall pipeline performance for the Pareto trade-off analysis in Section 4.5.4.

The evaluated RAGAS quality metrics are defined as follows:

Context Precision. Context Precision measures the retriever's ability to rank relevant chunks higher than irrelevant ones in the retrieved context [51]:

$$\text{Context Precision@}K = \begin{cases} \frac{\sum_{k=1}^{K} (\text{Precision@}k \times v_k)}{\sum_{k=1}^{K} v_k}, & \text{if } \sum_{k=1}^{K} v_k > 0 \\ 0, & \text{otherwise} \end{cases} \tag{16}$$

where K is the total number of retrieved context chunks, $v_k \in \{0, 1\}$ represents the binary relevance indicator at rank k, and $\text{Precision@}k = \frac{\text{true positives@}k}{\text{true positives@}k + \text{false positives@}k}$.

Context Recall. Context Recall evaluates whether the retrieved context covers all necessary reference statements required to answer the query [51]:

$$\text{Context Recall} = \frac{|\text{Ground Truth Sentences Attributed to Context}|}{|\text{Total Sentences in Ground Truth}|} \tag{17}$$

Because Context Recall requires verifiable reference statements, unanswerable and out-of-domain queries containing no factual claims are excluded from recall calculations and instead evaluated under the abstention framework detailed below (Equations 21 and 22).

Faithfulness. Faithfulness measures the factual consistency of the generated response against retrieved context R_q, penalizing ungrounded hallucinations [51]:

$$\text{Faithfulness} = \frac{|\text{Claims in Answer Inferred from Context}|}{|\text{Total Claims in Generated Answer}|} \tag{18}$$

Similarly, valid refusals contain no factual claims to evaluate and score 0 in standard Faithfulness. Therefore, unanswerable queries are excluded and evaluated separately via Refusal Accuracy (Equation 21).

Answer Relevancy. Answer Relevancy quantifies how directly the generated response addresses the user's initial prompt, penalizing incomplete answers or extraneous verbosity [51]:

$$\text{Answer Relevancy} = \frac{1}{N}\sum_{i=1}^{N}\frac{E(q)\cdot E(q_i^{\text{gen}})}{\|E(q)\|\|E(q_i^{\text{gen}})\|} \tag{19}$$

where N denotes the number of synthetic questions generated from the model's response by the evaluation judge (default $N = 3$), q_i^{gen} is the i-th generated question, and $E(\cdot)$ represents the dense embedding function instantiated via BGE-M3 [53] to compute cosine similarity against the original query q.

Composite Quality Score. The Composite Quality Score $Q(q) \in [0, 1]$ synthesizes the four individual quality dimensions from Equations 16–19 as a regularized harmonic mean:

$$Q(q) = \frac{4}{\frac{1}{\text{CP}(q)+\epsilon} + \frac{1}{\text{CR}(q)+\epsilon} + \frac{1}{\text{Faith}(q)+\epsilon} + \frac{1}{\text{AR}(q)+\epsilon}} \tag{20}$$

where $\text{CP}(q)$, $\text{CR}(q)$, $\text{Faith}(q)$, and $\text{AR}(q)$ denote Context Precision, Context Recall, Faithfulness, and Answer Relevancy for query q, respectively, and $\epsilon = 10^{-4}$ is a minimal smoothing parameter introduced to guarantee numerical stability and penalize

catastrophic failures in any single quality dimension. At the pipeline level, the Composite Quality score reported in evaluation tables and Pareto figures represents the arithmetic mean across all answerable queries, denoted as $\bar{Q} = \frac{1}{|Q_{\text{ans}}|} \sum_{q \in Q_{\text{ans}}} Q(q)$.

In addition to the standard RAGAS dimensions, Refusal Accuracy and False Refusal Rate (FRR) are evaluated to quantify the capability of each pipeline to handle non-answerable and out-of-domain queries. This dual-metric formulation addresses an inherent limitation of reference-free RAG evaluation suites [51]: when a pipeline correctly abstains from answering an unanswerable query by emitting the standardized refusal string, generation-level RAGAS metrics (particularly Faithfulness and Answer Relevancy) falsely score zero due to the absence of affirmative factual claims. To preserve evaluation integrity, RAGAS quality metrics (CP, CR, Faith, and AR) as well as the Composite Quality score $Q(q)$ (Equation 20) are computed exclusively over the answerable query partition Q_{ans}, while abstention behavior is evaluated using Equations 21 and 22.

Refusal Accuracy. Refusal Accuracy measures the proportion of unanswerable or out-of-domain queries (Q_{unans}) for which the pipeline correctly abstains by returning the predefined refusal response:

$$\text{Refusal Accuracy} = \frac{1}{|Q_{\text{unans}}|} \sum_{q \in Q_{\text{unans}}} \mathbb{I}_{\text{refusal}}(q) \tag{21}$$

where $\mathbb{I}_{\text{refusal}}(q) \in \{0, 1\}$ denotes an indicator function evaluating to 1 if the generated response matches canonical refusal semantics (in English for MuSiQue or Finnish for the enterprise dataset), and 0 otherwise.

False Refusal Rate (FRR). Conversely, an overly conservative pipeline may reject legitimate questions for which sufficient context is available. The False Refusal Rate measures the frequency of erroneous abstentions over the answerable query partition (Q_{ans}):

$$\text{FRR} = \frac{1}{|Q_{\text{ans}}|} \sum_{q \in Q_{\text{ans}}} \mathbb{I}_{\text{refusal}}(q) \tag{22}$$

Minimizing FRR while maximizing Refusal Accuracy ensures that hallucination mitigation does not compromise system availability for valid enterprise customer requests.

4.5.4 Pareto Frontier and Trade-off Equilibrium

To identify the Pareto frontier and quantify the trade-offs between response quality, operational expenditure, and execution speed, two practical efficiency ratios are formulated: Quality per Dollar (QPD) and Quality per Second (QPS). These composite

indicators capture the return on investment (ROI) for each optimization technique and multi-module combination across the experimental matrix:

$$\text{QPD} = \frac{\bar{Q}}{\text{Cost}_{1k}/1000} \tag{23}$$

$$\text{QPS} = \frac{\bar{Q}}{T_{\text{total},P_{50}}} \tag{24}$$

where $\bar{Q}$ is the configuration-level composite quality score, Cost_{1k} is the operational cost per 1,000 queries in USD from Equation 15, and $T_{\text{total},P_{50}}$ is the median end-to-end execution latency in seconds. Together, these ratios allow systematic identification of configurations that maximize generative quality while minimizing operational latency and cost overhead.

4.6 Threats to Validity

According to Wohlin et al. [54], empirical research in software engineering is subject to three primary categories of threats to validity: construct validity, internal validity, and external validity. Construct validity concerns whether the experimental instrumentation accurately measures the target theoretical concepts. Internal validity addresses whether observed performance differences are strictly attributable to the evaluated optimization methods rather than confounding system noise. Finally, external validity evaluates the degree to which the experimental findings generalize beyond the specific evaluation testbed.

Construct Validity. The RAGAS evaluation framework relies heavily on an LLM-as-a-judge. This dependency and the stochastic nature of LLMs introduce potential threats to validity, as the judge's evaluations may exhibit non-deterministic behavior. In addition, if the evaluation judge and the generative pipeline shared the same model family, the results would be susceptible to vendor self-preference bias. To mitigate these risks, an independent open-weights model from a distinct architectural family, Qwen3.8-27B, is selected as the evaluation judge and hosted locally via vLLM at zero temperature ($T = 0.0$) alongside the BGE-M3 embedding model to eliminate vendor self-preference bias and cloud API drift. Furthermore, to complement the model-based evaluation, deterministic token-level lexical overlap metrics (F_1 and Exact Match) are computed against gold standard reference answers.

Beyond the evaluation judge, the generation of the enterprise dataset also relies on LLMs—specifically Gemini 3.8 Flash, which was used to synthesize and verify the evaluation pairs. Because models from the same Gemini family are evaluated in the RAG pipeline, generator-evaluator alignment bias poses a potential threat to construct validity. To mitigate this concern, the dataset is grounded directly in crawled real-world documentation rather than unconstrained synthetic generation, and RAGAS quality

evaluation is conducted by an independent external model (the local Qwen3.8-27B judge). Furthermore, the human-annotated MuSiQue benchmark serves as an external baseline to verify that performance gains are not artifacts of the synthetic dataset.

Internal Validity. Distributed computer systems and network environments introduce numerous sources of latency variance, ranging from cold-start overheads to transient network jitter. To mitigate cold-start distortions, two unmeasured warm-up queries precede each experimental run. Although foundation model invocations traverse external network connections to GCP's global endpoints, network latency fluctuations are controlled by evaluating paired queries against the naive baseline (C_k vs. C_0) using the paired Wilcoxon signed-rank test. In addition, globally hosted foundation model services operate as "black boxes" from a client perspective, introducing latency variance driven by dynamic server-side loads and global cluster availability. These uncontrollable factors can introduce non-pipeline-related latency inflation that predominantly manifests in tail latencies. To mitigate this distortion, empirical results report both median (P_{50}) and 95th percentile (P_{95}) profiles, and benchmark executions are scheduled during off-peak hours to minimize queueing contention. Furthermore, fine-grained micro-phase telemetry isolates local retrieval and processing stages from network transit and remote generation durations.

By default, Gemini foundation models employ implicit prefix caching. To prevent unmanaged server-side cache hits from confounding experimental measurements, the benchmarking pipeline explicitly isolates caching benefits to M_3-enabled configurations, billing all prompt tokens at full uncached rates and enforcing zero cached tokens across all non-caching runs in reported accounting tables. In addition, to eliminate cold-start latency associated with initial cache allocation, two unmeasured pre-warming invocations precede each caching-enabled experimental run to ensure the cache is warm prior to recording measurements. Finally, application and database caches are explicitly cleared between distinct experimental configurations to prevent cross-configuration state leakage.

External Validity. Practically all of the systems introduced in Section 3 are implemented using a very homogeneous set of programming languages, relying mostly on Python. A primary motivation of this study is to examine how these optimization concepts generalize into enterprise-standard languages and frameworks, specifically Java and Spring AI. There is also a risk that the pre-training corpus of the underlying LLMs might include academic benchmarks used in this study, potentially enabling the model to answer queries based on memorized parametric knowledge rather than retrieved context. To mitigate this issue, the standardized MuSiQue dataset [50] is paired with a domain-specific synthetic customer support dataset to assess model generalizability.

Furthermore, evaluating hierarchical retrieval strategies such as Small-to-Big (M_1) on MuSiQue poses an external validity concern due to the dataset's native structure. MuSiQue passages are inherently short, pre-segmented Wikipedia paragraphs (typically

100–150 tokens). Reconstructing 1024-token parent context windows by aggregating passages sharing the same article title introduces a synthetic document hierarchy that may not fully reflect native document continuity.

5 Results

This chapter presents the empirical benchmark results across the sixteen evaluated RAG configurations ($C_0 \ldots C_{15}$) on the academic and enterprise corpora. Sections 5.1 and 5.2 evaluate execution latency and operational expenditures, while Section 5.3 investigates retrieval and generation quality. The resulting multi-objective trade-offs and empirical Pareto frontiers are analyzed in Section 5.4, followed by an executive delta synthesis in Section 5.5, with supplementary unconstrained enterprise evaluations detailed in Appendix D.

5.1 Latency Profiles and Computational Overhead

5.1.1 Academic Multi-Hop Benchmark (MuSiQue)

The latency profile on the MuSiQue benchmark largely aligns with theoretical expectations, while revealing several notable empirical patterns. Most notably, prompt caching (PC) yielded no statistically significant reduction in either TTFT or total end-to-end latency ($p \geq 0.05$), contrary to initial expectations. Latency increases under Small-to-Big (S2B) retrieval and Semantic Re-ranking (SR) were expected, as both optimization techniques introduce auxiliary computational stages into the RAG pipeline. In contrast, the latency penalty introduced by Model Cascading (MC) was markedly pronounced compared to S2B and SR, substantially inflating overall response times.

As shown in Table 5, the primary latency benefits of prompt caching emerged when integrated into multi-component architectures. For instance, compounding PC with the S2B + SR pipeline reduced median total latency from 1,733.5 ms to 1,527.0 ms. A comparable mitigation effect is observed in the S2B + PC + MC configuration relative to S2B + MC (decreasing median latency from 2,972.5 ms to 2,577.0 ms and P_{95} from 14.2 s to 4.6 s). Conversely, Model Cascading introduces substantial tail latency vulnerability (P_{95} exceeding 11 s to 30 s) whenever queries are routed to the frontier model, which incurs significantly longer token generation times.

Notably, S2B + PC was the only architectural configuration that achieved a statistically significant latency reduction relative to the naive baseline ($p < 0.001$), decreasing median execution time by 8.0% (from 1,359.0 ms to 1,250.0 ms) while nearly halving tail latency (P_{95} falling from 3,397.2 ms to 1,745.1 ms). Furthermore, across all evaluated pipelines, the gap between median TTFT and median total latency remained minimal (diverging by only 1–87 ms). This narrow delta is attributable to the concise nature of the generated answers (averaging 9–11 output tokens, as reported in Table 7), indicating that end-to-end execution duration is dominated almost entirely by the retrieval and prompt prefill phases rather than auto-regressive token generation.

Detailed component-wise execution latency breakdowns are provided in Appendix C (Figure C1a).

Table 5: End-to-end execution latency and Time-to-First-Token (TTFT) across 16 pipeline architectures on MuSiQue. Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline.

Pipeline	TTFT P_{50} (ms)	TTFT P_{95} (ms)	$T_{\text{total}} P_{50}$ (ms)	$T_{\text{total}} P_{95}$ (ms)	ΔT_{total} (%)
Baseline ($k = 0$)					
Naive RAG Baseline	1307.0	3392.5	1359.0	3397.2	–
Single-Factor Isolated ($k = 1$)					
S2B	1662.5	4303.9	1684.0	4313.7	+23.9%**
SR	1526.5	3214.2	1613.5	3221.2	+18.7%**
PC	1402.5	2430.5	1408.0	2429.5	+3.6%^{ns}
MC	2724.5	11407.9	2775.0	11986.4	+104.2%***
Pairwise Combinations ($k = 2$)					
S2B + SR	1671.0	7754.8	1733.5	7771.4	+27.6%***
S2B + PC	1239.0	1699.4	1250.0	1745.1	-8.0%***
S2B + MC	2967.0	13662.6	2972.5	14154.4	+118.7%***
SR + PC	1522.0	2318.6	1521.5	2318.6	+12.0%^{ns}
SR + MC	2730.5	9089.8	2732.5	9105.9	+101.1%***
PC + MC	2780.0	4504.8	2802.5	4504.2	+106.2%***
Higher-Order Combinations ($k \geq 3$)					
S2B + SR + PC	1524.5	2202.2	1527.0	2202.2	+12.4%^{ns}
S2B + SR + MC	2879.5	30752.5	2885.5	30883.4	+112.3%***
S2B + PC + MC	2571.5	4638.8	2577.0	4643.6	+89.6%***
SR + PC + MC	2916.5	5888.9	2931.0	5893.6	+115.7%***
S2B + SR + PC + MC	2807.0	4301.6	2808.0	4305.7	+106.6%***

Notes: P_{50} and P_{95} denote median and 95th percentile execution latencies. ΔT_{total} denotes percentage change relative to baseline. $^{*}p < 0.05$,** $p < 0.01$,*** $p < 0.001$, $^{\text{ns}}p \geq 0.05$ (paired Wilcoxon with Holm-Bonferroni correction vs. naive baseline).

5.1.2 Enterprise Production Corpus

The overall latency profiles on the enterprise closed corpus, presented in Table 6, largely mirror the empirical patterns observed on the academic benchmark. Most notably, prompt caching fails to confer execution speedups. Counter-intuitively, isolated PC statistically significantly inflates median latency by 21.8% ($p < 0.001$, rising from 1,407.0 ms to 1,714.0 ms), running contrary to theoretical latency expectations. Another unexpected observation is that isolated Small-to-Big retrieval (−4.3%) and Semantic Re-ranking (−1.2%) both exhibit slightly lower median execution latencies than the naive baseline, and even their joint integration (S2B + SR) introduces only a negligible 2.8% overhead (1,446.5 ms), none of which are statistically significant ($p \geq 0.05$).

Crucially, the severe tail latency spikes observed on the academic benchmark (where P_{95} frequently exceeded 11 s to 30 s under Model Cascading) are largely absent on the enterprise corpus. Across isolated and pairwise non-cascaded pipelines, tail latencies are much more stable, remaining tightly bounded between 2.2 s and 2.7 s (with only the higher-order S2B + SR + PC configuration reaching 4.5 s). This stability directly supports enterprise SLA compliance and perceived user responsiveness in production

Table 6: End-to-end execution latency and Time-to-First-Token (TTFT) across 16 pipeline architectures on Enterprise Closed. Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline.

Pipeline	TTFT P_{50} (ms)	TTFT P_{95} (ms)	$T_{\text{total}} P_{50}$ (ms)	$T_{\text{total}} P_{95}$ (ms)	ΔT_{total} (%)
Baseline ($k = 0$)					
Naive RAG Baseline	1135.0	1827.2	1407.0	2492.9	–
Single-Factor Isolated ($k = 1$)					
S2B	1117.0	1493.2	1347.0	2163.0	-4.3%^{ns}
SR	1200.0	1551.2	1389.5	2228.2	-1.2%^{ns}
PC	1562.5	1887.3	1714.0	2615.6	+21.8%***
MC	1715.0	3670.5	1943.5	4804.3	+38.1%***
Pairwise Combinations ($k = 2$)					
S2B + SR	1219.5	1698.4	1446.5	2240.7	+2.8%^{ns}
S2B + PC	1523.5	2232.6	1735.0	2686.5	+23.3%***
S2B + MC	1787.0	4339.8	2054.0	5956.2	+46.0%***
SR + PC	1672.0	2011.1	1836.5	2654.8	+30.5%***
SR + MC	1881.0	3764.3	2105.5	4606.1	+49.6%***
PC + MC	1974.0	4713.1	2150.5	4742.9	+52.8%***
Higher-Order Combinations ($k \geq 3$)					
S2B + SR + PC	1593.5	4083.6	1720.0	4510.3	+22.2%***
S2B + SR + MC	1927.5	3888.6	2127.5	5151.5	+51.2%***
S2B + PC + MC	2059.5	3163.5	2332.5	3849.2	+65.8%***
SR + PC + MC	1980.0	3299.7	2095.0	4057.1	+48.9%***
S2B + SR + PC + MC	2152.0	6482.9	2532.5	6531.1	+80.0%***

Notes: P_{50} and P_{95} denote median and 95th percentile execution latencies. ΔT_{total} denotes percentage change relative to baseline. $^{*}p < 0.05$,$^{**}p < 0.01$,$^{***}p < 0.001$, $^{\text{ns}}p \geq 0.05$ (paired Wilcoxon with Holm-Bonferroni correction vs. naive baseline).

environments. Furthermore, it indicates that the enterprise corpus does not induce the severe multi-hop retrieval bottlenecks characteristic of MuSiQue. A corresponding moderation is evident in median execution times under Model Cascading: while MC more than doubled median latency on MuSiQue (+104.2%), on the enterprise corpus isolated MC introduces a far more modest 38.1% latency overhead ($p < 0.001$, rising from 1,407.0 ms to 1,943.5 ms) because the routing tier retains most queries on faster models. Even the slowest evaluated pipeline (S2B + SR + PC + MC) tops out at 2,532.5 ms (+80.0%), remaining well below the 3-second mark observed across multi-stage configurations on MuSiQue. Overall, however, none of the evaluated optimization architectures provides a statistically significant latency improvement over the naive baseline on this corpus.

The substantially longer responses generated on the enterprise closed corpus (averaging 62–86 output tokens, representing a 7–9× increase over MuSiQue as reported in Tables 7 and 8) explain the wider divergence between median TTFT and total latency. While the gap on MuSiQue was restricted to tens of milliseconds (under 90 ms), the enterprise benchmark exhibits deltas on the order of 115–380 ms. This reflects a proportionally expanded auto-regressive token streaming phase (T_{stream}), demonstrating that generative synthesis accounts for a non-trivial fraction of total execution duration

when generating detailed enterprise summaries.

The corresponding fine-grained phase-wise latency distributions for the enterprise closed corpus are illustrated in Appendix C (Figure C1b).

5.2 Operational Expenditure and Token Dynamics

5.2.1 Academic Multi-Hop Benchmark (MuSiQue)

Table 7 details the operational expenditure and token dynamics across all 16 pipeline architectures on MuSiQue, underscoring the decisive economic impact of prompt caching. Across all cached configurations, exactly 7,157 prompt tokens are served directly from the context cache at a 90% tariff discount, leaving only 842–1,037 tokens billed at full uncached prefill rates. Consequently, in isolation, PC achieves a statistically significant 80.0% cost reduction ($p < 0.001$) relative to the uncached naive baseline, and these economic savings persist when integrated with multi-stage retrieval methods such as S2B or SR. The negligible cost delta between the naive baseline and isolated S2B (+1.6%) is attributable to the fact that the native paragraph boundaries in MuSiQue closely match the S2B child chunk sizing, resulting in virtually identical prompt token volumes.

The most pronounced cost escalations are observed across configurations incorporating Semantic Re-ranking. This stems from the auxiliary API fees of the cross-encoder re-ranking service, which charges a fixed rate per scored document ($0.00010 per document, detailed in Table B1), rendering neural re-ranking economically demanding over larger candidate pools. Interestingly, Model Cascading incurs only a moderate cost increase (+24.8% in isolation), even though Gemini 3.1 Pro carries higher token pricing (Table B1) and handled 77% of all queries in isolation (and 73%–77% across cascaded pipelines). The highest overall expenditure occurs when all non-caching optimizations are compounded (S2B + SR + MC), reaching $20.346 per 1,000 queries (+68.4% over baseline). However, introducing prompt caching into this configuration (S2B + SR + PC + MC) curtails expenditure to $8.389, transforming an otherwise severe cost regression into a 30.6% net saving relative to the naive baseline.

Detailed phase-wise monetary cost decompositions across uncached prompt prefill, context cache lookups, generative completion tokens, auxiliary re-ranking services, and router classifications are provided in Appendix C (Figure C2a).

5.2.2 Enterprise Production Corpus

The structural divergences between the two corpora are clearly visible in the prompt and completion token volumes reported in Tables 8 and 7, driving unoptimized baseline expenditure higher on the enterprise dataset ($13.580 versus $12.082 per 1,000 queries). In isolation, prompt caching achieves a statistically significant 70.7% expenditure reduction ($p < 0.001$, falling to $3.975 per 1,000 queries). The slightly narrower cost reduction relative to MuSiQue's 80.0% is driven by larger chunk sizes in the enterprise closed corpus, which expand the volume of uncached prompt tokens

Table 7: Operational cost and token consumption breakdown across 16 pipeline architectures on MuSiQue. Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline.

Pipeline	Uncached Prompt Tok	Cached Prompt Tok	Output Tokens	Cost / 1k ($)	Δ Cost (%)
Baseline ($k = 0$)					
Naive RAG Baseline	8,001	0	9	$12.082	–
Single-Factor Isolated ($k = 1$)					
S2B	8,132	0	9	$12.278	+1.6%**
SR	8,030	0	10	$17.131	+41.8%***
PC	842	7,157	9	$2.420	-80.0%***
MC	8,001	0	10	$15.079	+24.8%***
Pairwise Combinations ($k = 2$)					
S2B + SR	8,196	0	10	$17.388	+43.9%***
S2B + PC	973	7,157	9	$2.612	-78.4%***
S2B + MC	8,132	0	10	$14.977	+24.0%***
SR + PC	871	7,157	9	$7.466	-38.2%***
SR + MC	8,030	0	10	$19.914	+64.8%***
PC + MC	842	7,157	10	$3.088	-74.4%***
Higher-Order Combinations ($k \geq 3$)					
S2B + SR + PC	1,037	7,157	9	$7.715	-36.1%***
S2B + SR + MC	8,196	0	10	$20.346	+68.4%***
S2B + PC + MC	973	7,157	10	$3.374	-72.1%***
SR + PC + MC	871	7,157	11	$8.132	-32.7%***
S2B + SR + PC + MC	1,037	7,157	10	$8.389	-30.6%***

Notes: Per-query sample token averages. Cost normalized per 1,000 queries based on Google Cloud Gemini enterprise tariffs. ΔCost denotes percentage change relative to baseline. $^{*}p < 0.05$,** $p < 0.01$,*** $p < 0.001$, $^{\text{ns}}p \geq 0.05$ (paired Wilcoxon with Holm-Bonferroni correction vs. naive baseline).

(1,446–2,034 dynamic tokens versus 842–1,037 on MuSiQue) while the cached prefix remains identical (7,157 tokens).

In addition, Model Cascading demonstrates a notable economic inversion on the enterprise corpus: rather than increasing expenditure as in multi-hop reasoning (+24.8%), isolated MC yields a statistically significant 34.2% cost reduction ($p < 0.001$, falling to $8.942 per 1,000 queries). This economy arises because 49% of all incoming queries were successfully resolved by the economical Gemini 3.5 Flash-Lite tier rather than escalating to Gemini 3.5 Flash or Gemini 3.1 Pro. Furthermore, routing to lower-tier models effectively mitigated the cost overhead of multi-stage retrieval, reducing the expenditure of the S2B + SR pipeline from $19.196 to $14.082 per 1,000 queries (a 26.6% savings) and achieving economic relief comparable to prompt caching.

Compounding prompt caching with model cascading (PC + MC) delivers the most substantial economic savings in Table 8, driving operational expenditure down to $2.543 per 1,000 queries (an 81.3% reduction relative to the baseline). At the opposite extreme, combining Small-to-Big retrieval with Semantic Re-ranking (S2B +

Table 8: Operational cost and token consumption breakdown across 16 pipeline architectures on Enterprise Closed. Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline.

Pipeline	Uncached Prompt Tok	Cached Prompt Tok	Output Tokens	Cost / 1k ($)	Δ Cost (%)
Baseline ($k = 0$)					
Naive RAG Baseline	8,605	0	75	$13.580	–
Single-Factor Isolated ($k = 1$)					
S2B	9,193	0	76	$14.471	+6.6%***
SR	8,795	0	77	$18.881	+39.0%***
PC	1,446	7,157	81	$3.975	-70.7%***
MC	8,605	0	66	$8.942	-34.2%***
Pairwise Combinations ($k = 2$)					
S2B + SR	9,024	0	73	$19.196	+41.4%***
S2B + PC	2,034	7,157	86	$4.900	-63.9%***
S2B + MC	9,193	0	73	$9.290	-31.6%***
SR + PC	1,636	7,157	84	$9.282	-31.6%***
SR + MC	8,795	0	65	$13.805	+1.7%^{ns}
PC + MC	1,446	7,157	68	$2.543	-81.3%***
Higher-Order Combinations ($k \geq 3$)					
S2B + SR + PC	1,865	7,157	77	$9.561	-29.6%***
S2B + SR + MC	9,024	0	62	$14.082	+3.7%^{ns}
S2B + PC + MC	2,034	7,157	75	$3.201	-76.4%***
SR + PC + MC	1,636	7,157	71	$7.766	-42.8%***
S2B + SR + PC + MC	1,865	7,157	66	$7.922	-41.7%***

Notes: Per-query sample token averages. Cost normalized per 1,000 queries based on Google Cloud Gemini enterprise tariffs. ΔCost denotes percentage change relative to baseline. $^{*}p < 0.05$,$^{**}p < 0.01$,$^{***}p < 0.001$, $^{ns}p \geq 0.05$ (paired Wilcoxon with Holm-Bonferroni correction vs. naive baseline).

SR) represents the most expensive pipeline at $19.196 per 1,000 queries (+41.4%). However, even this demanding multi-stage retrieval architecture can be rendered economically sustainable by introducing joint caching and model routing (S2B + SR + PC + MC), which curtails expenditure down to $7.922 and yields a net 58.7% cost reduction compared to S2B + SR alone.

The corresponding component-wise operational expenditure breakdown per 1,000 queries for the enterprise closed corpus is illustrated in Appendix C (Figure C2b).

5.3 Retrieval and Generation Quality

5.3.1 Academic Multi-Hop Benchmark (MuSiQue)

As shown in Table 9, none of the architectural optimizations yielded a statistically significant improvement in response quality over the naive baseline ($p \geq 0.05$ across all configurations). This performance ceiling stems primarily from the intrinsic compositional structure of the MuSiQue benchmark. In particular, 3- and 4-hop queries exhibit high multi-hop reasoning complexity where the raw input query lacks

sufficient semantic overlap to retrieve all disjoint, intermediate supporting passages within a single-shot dense retrieval step. Furthermore, under a constrained context window of top-$k = 5$, a 4-hop question requires an 80% retrieval precision (4 out of 5 slots) simply to assemble the requisite context, leaving virtually no tolerance for distracting passages.

Table 9: Retrieval, generation, and multi-hop reasoning quality metrics across 16 pipeline architectures on the MuSiQue benchmark ($N = 100$). Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline.

Pipeline	Context Precision	Context Recall	Faithfulness	Answer Relevancy	Exact Match / Token F1 (%)	Refusal Acc. / FRR (%)	Composite Quality Q
Baseline ($k = 0$)							
Naive RAG Baseline	0.505	0.562	0.588	0.242	41.0% / 45.1%	65.0% / 42.5%	**0.2104**
Single-Factor Isolated ($k = 1$)							
S2B	0.454	0.575	0.583	0.241	37.0% / 44.4%	65.0% / 42.5%	0.1851^{ns}
SR	0.456	0.463	0.623	0.221	37.0% / 42.1%	70.0% / 48.8%	0.1735^{ns}
PC	0.472	0.562	0.584	0.244	39.0% / 43.8%	70.0% / 45.0%	0.1856^{ns}
MC	0.485	0.550	0.581	0.187	42.0% / 45.6%	90.0% / 56.2%	0.1789^{ns}
Pairwise Combinations ($k = 2$)							
S2B + SR	0.425	0.487	0.623	0.246	34.0% / 41.8%	70.0% / 46.2%	0.1995^{ns}
S2B + PC	0.486	0.575	0.560	0.256	39.0% / 46.3%	65.0% / 42.5%	0.2006^{ns}
S2B + MC	0.456	0.575	0.585	0.174	37.0% / 41.9%	85.0% / 60.0%	0.1566^{ns}
SR + PC	0.463	0.450	0.623	0.218	37.0% / 42.2%	70.0% / 51.2%	0.1697^{ns}
SR + MC	0.457	0.450	0.598	0.175	36.0% / 40.2%	80.0% / 61.3%	0.1575^{ns}
PC + MC	0.470	0.562	0.569	0.185	39.0% / 42.7%	85.0% / 57.5%	0.1600^{ns}
Higher-Order Combinations ($k \geq 3$)							
S2B + SR + PC	0.441	0.500	0.607	0.239	35.0% / 41.2%	70.0% / 45.0%	0.2069^{ns}
S2B + SR + MC	0.430	0.487	0.635	0.181	33.0% / 38.8%	75.0% / 58.8%	0.1834^{ns}
S2B + PC + MC	0.478	0.562	0.579	0.166	38.0% / 41.6%	90.0% / 62.5%	0.1664^{ns}
SR + PC + MC	0.451	0.463	0.588	0.141	35.0% / 36.8%	85.0% / 68.8%	0.1209^{ns}
S2B + SR + PC + MC	0.418	0.481	0.619	0.167	37.0% / 41.1%	80.0% / 61.3%	0.1486^{ns}

Notes: Evaluated via RAGAS [51] on answerable queries. EM and Token F1 measure lexical overlap. Refusal Acc. denotes abstention on unanswerable queries; FRR measures false refusals on answerable queries. Composite Quality $Q \in [0, 1]$ represents regularized harmonic mean (Eq. 20). Best Q in bold. $^{*}p < 0.05$,$^{**}p < 0.01$,$^{***}p < 0.001$, $^{ns}p \geq 0.05$ (paired Wilcoxon with Holm-Bonferroni correction vs. naive baseline).

A notable behavioral pattern emerges in Model Cascading, which demonstrates substantially higher refusal accuracy on unanswerable queries than non-cascaded architectures. In isolated single-factor evaluations, MC achieved a refusal accuracy of 90.0% compared to 65.0%–70.0% across other configurations. However, this apparent conservatism comes at the cost of an elevated False Refusal Rate (FRR, rising to 56.2% in isolation and up to 68.8% in SR + PC + MC). This reveals that rather than displaying genuine multi-hop reasoning discernment, the frontier model tier suffered from systematic over-refusal. Specifically, raw query execution logs indicate that the model refused to answer 65% of all benchmark queries under SR + MC, and 72% under SR + PC + MC. Consequently, as reflected by the lack of statistically significant divergence across Composite Quality (Q), these conservative refusal dynamics did not translate into meaningful quality improvements over the baseline.

Exact Match (EM) and Token F1 scores remain low and tightly bounded across all configurations, spanning 33.0%–42.0% for EM and 36.8%–46.3% for Token F1. Model

Cascading achieves the highest EM score of 42.0% alongside the second-highest F1 score (45.6%), whereas S2B + PC delivers the highest overall F1 score (46.3%). This elevation in MC's lexical overlap is primarily an artifact of refusal scoring, as correct abstentions on unanswerable queries are rewarded with a score of 1.0 in both metrics. Conversely, this high propensity to abstain severely penalizes answerable queries: MC's elevated False Refusal Rate causes Answer Relevancy to plummet from the baseline's 0.242 to 0.141–0.187 across cascaded configurations. Furthermore, while isolated optimizations occasionally improve individual dimensions—such as Semantic Re-ranking increasing Faithfulness from 0.588 to 0.623 at the expense of Context Recall (0.562 to 0.463)—the penalizing sensitivity of the regularized harmonic mean (Equation 20) causes these localized weaknesses to depress Composite Quality (Q) overall.

Configuration-level multi-dimensional response quality profiles spanning Context Precision, Context Recall, Faithfulness, and Answer Relevancy across architectural tiers are visualized as tier-stratified lollipop plots in Appendix C (Figure C3).

5.3.2 Enterprise Production Corpus

As reported in Table 10, five of the evaluated optimization architectures achieved modest increases in Composite Quality (Q) over the naive baseline (0.6209), although none reached statistical significance ($p \geq 0.05$). The highest overall quality was attained by Small-to-Big retrieval paired with prompt caching (S2B + PC), which established the global benchmark maximum at $Q = 0.6711$ without exhibiting localized regressions across any individual RAGAS dimension. Notably, all 16 evaluated architectures achieved flawless Refusal Accuracy (100.0%) on unanswerable out-of-domain queries, in stark contrast to the 65.0%–90.0% range on MuSiQue. Within this deterministic rejection baseline, S2B + PC maintained a low False Refusal Rate of 10.0% (tied for second lowest behind S2B + SR + PC at 8.8%), confirming reliable abstention dynamics without excessive over-refusal. Furthermore, whereas Semantic Re-ranking severely degraded Context Recall on MuSiQue, enterprise Context Recall remained exceptionally robust and invariant across all configurations, spanning a tight band of 0.819 to 0.850.

The lowest-performing configurations on the enterprise corpus are predominantly associated with Model Cascading. In single-factor evaluations, isolated MC achieved the lowest score among isolated techniques ($Q = 0.5974$), driven by slight reductions in Answer Relevancy (0.651) and Faithfulness (0.826) relative to the baseline. Across all evaluated pipelines, the worst-performing individual configuration is Semantic Re-ranking combined with Model Cascading (SR + MC), which registered a Composite Quality of $Q = 0.5418$ (a 12.7% regression relative to the naive baseline). This deterioration stems from notable regressions in both Faithfulness (falling to 0.780) and Answer Relevancy (falling to 0.628), alongside the benchmark's highest False Refusal Rate (17.5%). Interestingly, this quality penalty occurs despite Context Precision reaching the global benchmark maximum (0.683), demonstrating that while cross-encoder re-ranking curates highly precise context passages, routing simpler

Table 10: Retrieval, generation, and out-of-domain rejection quality metrics across 16 pipeline architectures on the Enterprise Knowledge Corpus ($N = 100$). Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline.

Pipeline	Context Precision	Context Recall	Faithfulness	Answer Relevancy	Refusal Acc. / FRR (%)	Composite Quality Q
Baseline ($k = 0$)						
Naive RAG Baseline	0.601	0.844	0.844	0.666	100.0% / 11.2%	0.6209
Single-Factor Isolated ($k = 1$)						
S2B	0.655	0.850	0.863	0.661	100.0% / 11.2%	0.6449^{ns}
SR	0.676	0.831	0.845	0.678	100.0% / 13.8%	0.6137^{ns}
PC	0.624	0.838	0.836	0.664	100.0% / 12.5%	0.6189^{ns}
MC	0.609	0.844	0.826	0.651	100.0% / 15.0%	0.5974^{ns}
Pairwise Combinations ($k = 2$)						
S2B + SR	0.616	0.827	0.854	0.671	100.0% / 11.2%	0.5813^{ns}
S2B + PC	0.666	0.844	0.886	0.693	100.0% / 10.0%	$\mathbf{0.6711}^{\text{ns}}$
S2B + MC	0.661	0.844	0.858	0.677	100.0% / 11.2%	0.6587^{ns}
SR + PC	0.671	0.829	0.878	0.681	100.0% / 11.2%	0.6299^{ns}
SR + MC	0.683	0.819	0.780	0.628	100.0% / 17.5%	0.5418^{ns}
PC + MC	0.624	0.838	0.831	0.651	100.0% / 13.8%	0.6085^{ns}
Higher-Order Combinations ($k \geq 3$)						
S2B + SR + PC	0.629	0.827	0.865	0.692	100.0% / 8.8%	0.6134^{ns}
S2B + SR + MC	0.632	0.827	0.808	0.666	100.0% / 12.5%	0.5808^{ns}
S2B + PC + MC	0.659	0.848	0.838	0.691	100.0% / 10.0%	0.6411^{ns}
SR + PC + MC	0.672	0.825	0.798	0.661	100.0% / 13.8%	0.5969^{ns}
S2B + SR + PC + MC	0.628	0.827	0.852	0.666	100.0% / 12.5%	0.6050^{ns}

Notes: Evaluated via RAGAS [51] on answerable queries. Refusal Acc. denotes abstention on unanswerable out-of-domain (OOD) queries; FRR measures false refusals on answerable queries. Composite Quality $Q \in [0, 1]$ represents regularized harmonic mean (Eq. 20). Best Q in bold. $^{*}p < 0.05$,$^{**}p < 0.01$,$^{***}p < 0.001$, $^{\text{ns}}p \geq 0.05$ (paired Wilcoxon with Holm-Bonferroni correction vs. naive baseline).

queries to smaller models can compromise synthesis fidelity in specialized enterprise domains.

Exact Match and Token F1 scores are intentionally omitted from the enterprise evaluation because the structural format of the responses differs fundamentally from the academic benchmark. In the enterprise domain, generated responses consist of multi-sentence procedural explanations averaging 20–35 words (corresponding to 62–86 tokens as reported in Table 8). In contrast, MuSiQue reference answers predominantly comprise concise entity names or short factual phrases (averaging 1–3 words). Under such generative multi-sentence synthesis, surface-level lexical overlap metrics become uninformative, dropping artificially close to zero despite high semantic precision. Consequently, response quality is comprehensively characterized via the four RAGAS dimensions. The corresponding multi-dimensional quality profiles across architectural tiers are visualized as tier-stratified lollipop plots in Appendix C (Figure C4).

Table 11: Multi-objective Pareto efficiency and 3D non-domination trade-off metrics across 16 pipeline architectures on MuSiQue.

Pipeline	Composite Quality Q	Cost / 1k ($)	$T_{\text{total}}P_{50}$ (s)	QPD	QPS	Pareto Optimal
Baseline ($k = 0$)						
Naive RAG Baseline	0.2104	$12.082	1.36	17	0.15	**Yes**
Single-Factor Isolated ($k = 1$)						
S2B	0.1851	$12.278	1.68	15	0.11	No
SR	0.1735	$17.131	1.61	10	0.11	No
PC	0.1856	$2.420	1.41	77	0.13	**Yes**
MC	0.1789	$15.079	2.77	12	0.06	No
Pairwise Combinations ($k = 2$)						
S2B + SR	0.1995	$17.388	1.73	11	0.12	No
S2B + PC	0.2006	$2.612	1.25	77	0.16	**Yes**
S2B + MC	0.1566	$14.977	2.97	10	0.05	No
SR + PC	0.1697	$7.466	1.52	23	0.11	No
SR + MC	0.1575	$19.914	2.73	8	0.06	No
PC + MC	0.1600	$3.088	2.80	52	0.06	No
Higher-Order Combinations ($k \geq 3$)						
S2B + SR + PC	0.2069	$7.715	1.53	27	0.14	**Yes**
S2B + SR + MC	0.1834	$20.346	2.89	9	0.06	No
S2B + PC + MC	0.1664	$3.374	2.58	49	0.06	No
SR + PC + MC	0.1209	$8.132	2.93	15	0.04	No
S2B + SR + PC + MC	0.1486	$8.389	2.81	18	0.05	No

Notes: Composite Quality $Q \in [0, 1]$ (Eq. 20). QPD = Q/Cost_{1k} and QPS = Q/T_{total} represent resource efficiency ratios. Pareto optimality denotes 3D non-domination ($\max Q$, $\min \text{Cost}$, $\min T_{\text{total}}$). Bold indicates Pareto-optimal configuration.

5.4 Trade-off Equilibrium and Pareto Frontier

5.4.1 Academic Multi-Hop Benchmark (MuSiQue)

As summarized in Table 11, four architectures define the non-dominated empirical Pareto frontier on MuSiQue: the naive baseline, PC, S2B + PC, and S2B + SR + PC. Across all Pareto-optimal configurations, non-domination is primarily propelled by steep operational cost reductions alongside well-preserved Composite Quality (Q). For each non-baseline Pareto configuration, Quality per Dollar (QPD) and Quality per Second (QPS) demonstrate superior efficiency trade-offs relative to non-cached alternatives. Conversely, the high auxiliary API charges of Semantic Re-ranking and the severe latency overhead of Model Cascading represent the primary causes of sub-optimality in dominated pipelines, whereas prompt caching acts as the critical catalyst for Pareto efficiency. For instance, compounding PC with the S2B + SR pipeline elevates its QPD by 145.5% (from 11 to 27) and QPS by 16.7% (from 0.12 to 0.14). In contrast, introducing MC into the S2B + SR pipeline degrades both efficiency ratios—reducing QPD from 11 to 9 and halving QPS from 0.12 to 0.06—driven primarily by a 67.1% surge in median latency and a moderate 17.0% cost increase.

The structural divergence between Model Cascading and non-cascaded architectures is visually evident in Figure 1(a). All MC configurations cluster distinctly in the

high-latency regime (T_{total} P_{50} $>$ 2.5 s along the right-hand boundary), whereas non-cascaded pipelines remain confined to lower latencies (1.25–1.75 s). Furthermore, Composite Quality remains constrained within a narrow band across all variations ($Q \approx 0.15$–0.21), underscoring the limited impact of model routing on retrieval-constrained reasoning. In contrast, Figure 1(b) reveals that operational expenditures do not exhibit binary clustering, but are distributed continuously across the \$2.42 to \$20.35 range depending directly on the inclusion of prompt caching and cross-encoder re-ranking.

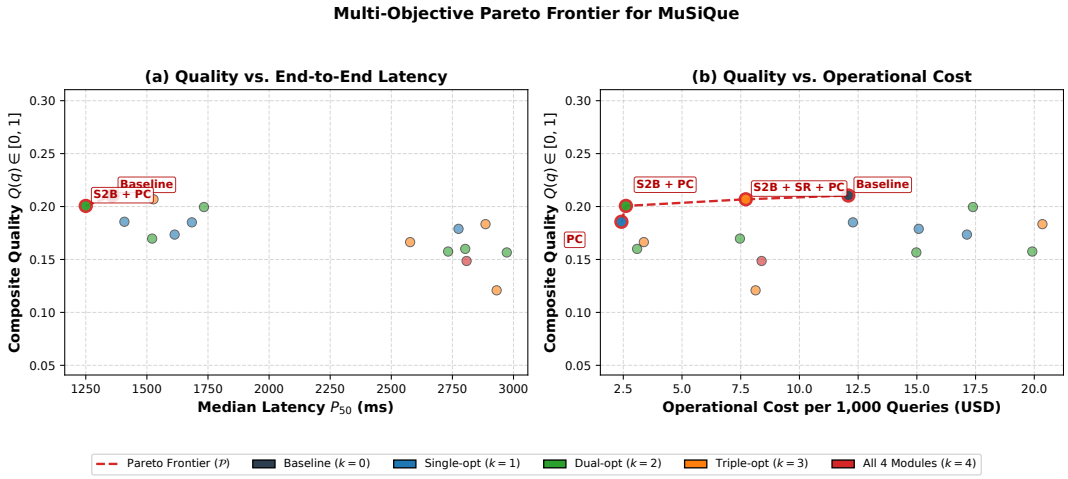


Figure 1: Empirical Pareto frontier of Composite Quality (Q) versus median end-to-end execution latency (T_{total} P_{50}) and operational expenditure ($\text{Cost}_{1\text{k}}$) on the academic multi-hop benchmark (MuSiQue). Solid lines delineate the non-dominated empirical Pareto frontier across configurations.

5.4.2 Enterprise Production Corpus

As summarized in Table 12, six architectures define the non-dominated empirical Pareto frontier on the enterprise closed corpus: the naive baseline, isolated S2B, PC, S2B + PC, PC + MC, and S2B + PC + MC. In this setting, non-domination is primarily governed by operational cost efficiency (QPD), though execution throughput (QPS) also serves as a key driver for the naive baseline and isolated S2B. The global maximum in Quality per Dollar is achieved by the most economical architecture, PC + MC (QPD = 239), whereas isolated Small-to-Big retrieval provides the benchmark peak in Quality per Second (QPS = 0.48). Only one Pareto-optimal architecture exhibits a noticeable quality regression relative to the naive baseline, with PC + MC incurring a minor 2.0% drop ($Q = 0.6085$ versus 0.6209). Furthermore, although isolated Model Cascading achieves neither superior QPD nor QPS on its own, compounding it with S2B + PC preserves high Composite Quality ($Q = 0.6411$) while substantially boosting QPD from 137 to 200. Crucially, three architectures maintain Pareto optimality across both evaluated benchmark corpora: the naive baseline, PC, and S2B + PC.

Figure 2 visualizes the multi-objective Pareto trade-off space, projecting Composite

Table 12: Multi-objective Pareto efficiency and 3D non-domination trade-off metrics across 16 pipeline architectures on Enterprise Closed.

Pipeline	Composite Quality Q	Cost / 1k ($)	$T_{\text{total}} P_{50}$ (s)	QPD	QPS	Pareto Optimal
Baseline ($k = 0$)						
Naive RAG Baseline	0.6209	$13.580	1.41	46	0.44	**Yes**
Single-Factor Isolated ($k = 1$)						
S2B	0.6449	$14.471	1.35	45	0.48	**Yes**
SR	0.6137	$18.881	1.39	33	0.44	No
PC	0.6189	$3.975	1.71	156	0.36	**Yes**
MC	0.5974	$8.942	1.94	67	0.31	No
Pairwise Combinations ($k = 2$)						
S2B + SR	0.5813	$19.196	1.45	30	0.40	No
S2B + PC	0.6711	$4.900	1.74	137	0.39	**Yes**
S2B + MC	0.6587	$9.290	2.05	71	0.32	No
SR + PC	0.6299	$9.282	1.84	68	0.34	No
SR + MC	0.5418	$13.805	2.11	39	0.26	No
PC + MC	0.6085	$2.543	2.15	239	0.28	**Yes**
Higher-Order Combinations ($k \geq 3$)						
S2B + SR + PC	0.6134	$9.561	1.72	64	0.36	No
S2B + SR + MC	0.5808	$14.082	2.13	41	0.27	No
S2B + PC + MC	0.6411	$3.201	2.33	200	0.27	**Yes**
SR + PC + MC	0.5969	$7.766	2.10	77	0.28	No
S2B + SR + PC + MC	0.6050	$7.922	2.53	76	0.24	No

Notes: Composite Quality $Q \in [0, 1]$ (Eq. 20). QPD = Q/Cost_{1k} and QPS = Q/T_{total} represent resource efficiency ratios. Pareto optimality denotes 3D non-domination (max Q, min Cost, min T_{total}). Bold indicates Pareto-optimal configuration.

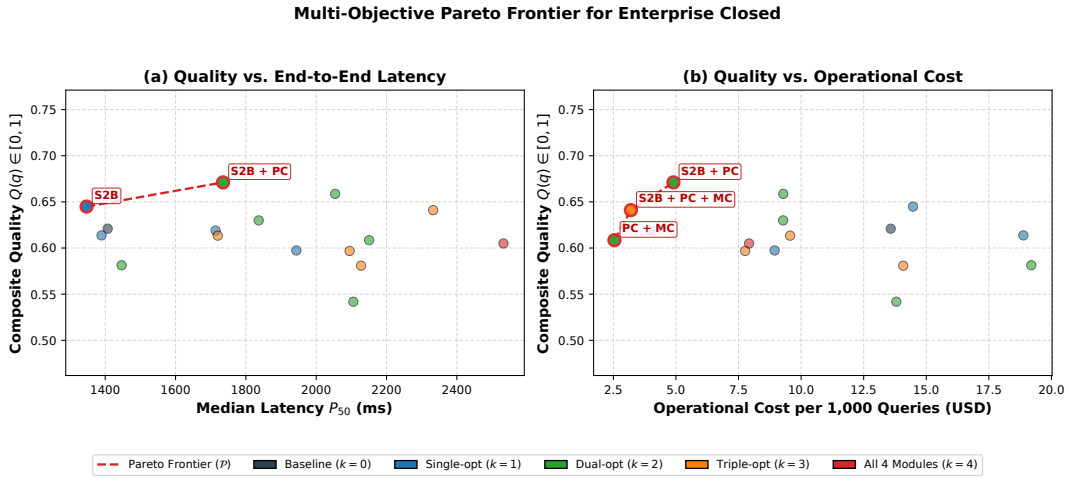


Figure 2: Empirical Pareto frontier of Composite Quality (Q) versus median end-to-end execution latency (T_{total} P_{50}) and operational expenditure (Cost_{1k}) on the enterprise closed benchmark corpus. Solid lines delineate the non-dominated empirical Pareto frontier across configurations.

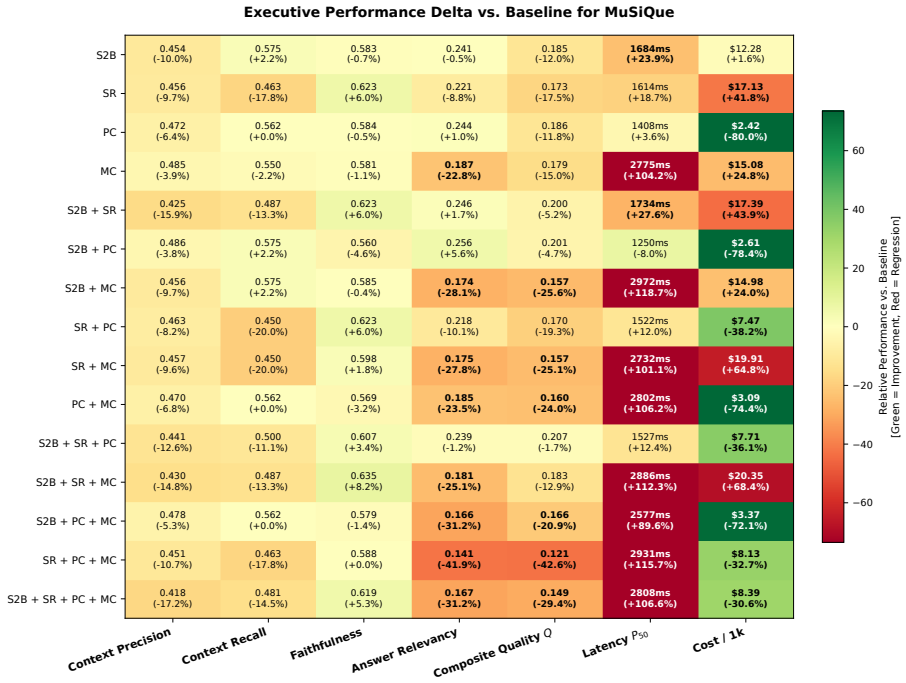
Quality (Q) against median execution latency in panel (a) and against operational expenditure in panel (b). As evident in Figure 2(b), the evaluated architectures segregate into four discrete operational cost clusters. The primary cluster ($2.50–$5.00 per 1,000 queries) encompasses all cost-efficient Pareto-optimal configurations (PC + MC, S2B + PC + MC, PC, and S2B + PC), demonstrating that prompt caching is essential for frontier positioning. The remaining pipelines stratify into three higher bands ($7.50–$10.00, $13.50–$14.50, and $18.80–$19.20), driven progressively by multi-stage retrieval overhead and uncached API charges. In contrast, execution latencies in Figure 2(a) are more dispersed. A compact low-latency cluster occupies the 1,340–1,450 ms window (encompassing S2B, SR, and the baseline), whereas pipelines incorporating model routing or prompt cache lookups shift toward higher latency regimes (1,700–2,530 ms).

5.5 Executive Performance Delta Synthesis

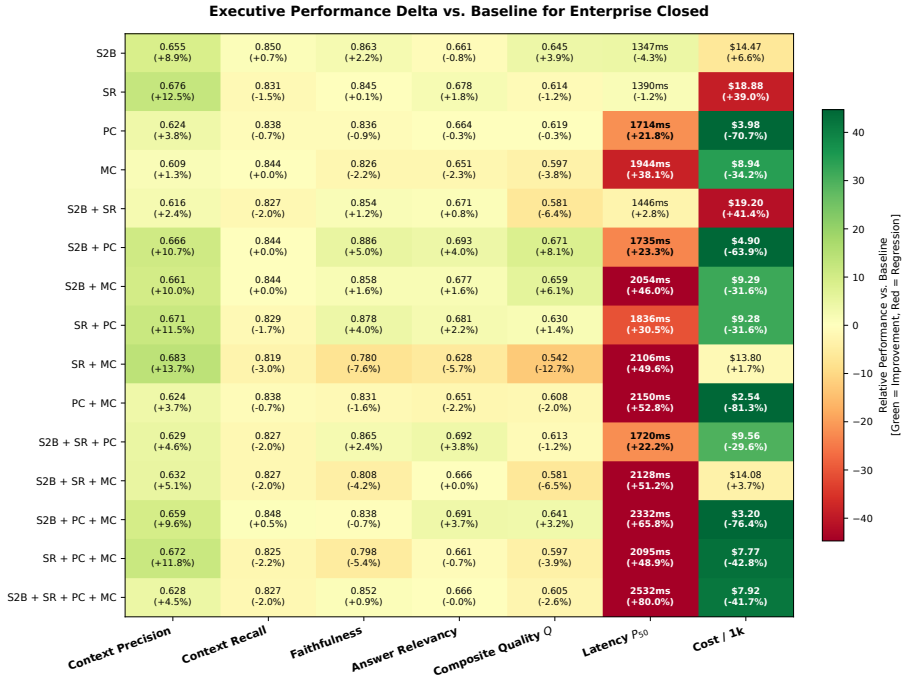
To provide a consolidated cross-dimensional view of all 15 architectural optimization treatments relative to the unoptimized naive baseline, Figures 3a and 3b visualize relative percentage improvements (green) and regressions (red) across all four RAGAS quality dimensions, Composite Quality (Q), median end-to-end latency (P_{50}), and operational expenditure per 1,000 queries. The performance delta heatmaps demonstrate that surpassing the generative quality of the naive baseline represents a substantial empirical challenge. On the MuSiQue benchmark, no architectural optimization outperformed the baseline in Composite Quality, whereas on the enterprise closed corpus, five configurations achieved positive quality deltas, capped at a modest global maximum of +8.1% (attained by S2B + PC). A comparable asymmetry characterizes execution latencies. While S2B + PC achieved a statistically significant 8.0% speedup on MuSiQue, latency reductions were otherwise rare, and multi-stage pipelines frequently incurred severe latency penalties that doubled execution times (+80% to +119%). Consequently, the most decisive empirical benefit of modular RAG optimization lies in operational expenditure reduction, where prompt caching consistently unlocks substantial cost savings (ranging from 29.6% up to 81.3%) without sacrificing output quality.

While the primary enterprise analysis of this thesis focuses strictly on the controlled closed corpus to ensure direct structural comparability with MuSiQue, exploratory supplementary evaluations were also conducted on the partner enterprise's unconstrained raw knowledge corpus (detailed in Appendix D). This noisy search space serves as an exploratory operational stress-test reflecting unfiltered production environments. Notably, the architectural Pareto frontier proves exceptionally robust across both settings, with the exact same six configurations defining the empirical trade-off surface. However, whereas the controlled candidate density of the closed corpus establishes a performance ceiling where multi-stage retrieval gains remain statistically non-significant, the extensive distractor noise of the unconstrained corpus causes naive baseline quality to deteriorate ($Q = 0.5022$). In this demanding setting, multi-stage retrieval acts as a critical noise-filtering mechanism: compounding Small-to-Big re-

trieval with Semantic Re-ranking delivers statistically significant quality improvements over the naive baseline (e.g., S2B + SR achieving $Q = 0.6433$, $p < 0.01$, and S2B + SR + PC reaching $Q = 0.6466$, $p < 0.01$). This confirms that while modular retrieval optimizations introduce marginal returns on pre-filtered datasets, their capacity to mitigate retrieval noise becomes decisive in real-world enterprise repositories.



(a) Academic multi-hop benchmark (MuSiQue)



(b) Enterprise closed benchmark corpus

Figure 3: Executive performance delta heatmaps relative to the naive baseline on (a) MuSiQue and (b) Enterprise Closed. Cells display absolute metrics alongside relative percentage changes (±%). Diverging color gradient highlights performance improvements (green) versus cost/latency regressions (red).

6 Discussion

6.1 The Prompt Caching Paradox: Latency Inversion versus Tail-Latency Protection

Promt caching did not provided any latency benefits in either of the tested corpora, which is a clearly against the acedemic expectations of the Chapter 3, and even increased the latencies in some of the tested configurations. <lisää jotain lähteitä tähän ja vertaile tuloksia>

In the enterprise closed corpus, standalone prompt caching median latency were 21.8% higher than more heavier S2B + SR configuration, which does computationally heavy reranking and child-to-parent expansions, and thus could be expected to be slower. There are couple of reasons for that. The first reason is that prefil phrase have extra $\tilde{5}$00ms delay that comes from the lookup and extra network calls for the KV cache. When the streaming part is indeed faster (147ms for baseline and 105ms for prompt caching), this does not help to offset the increased latency from the prefil. The second reason is that the cached context were too small to take full advantage of if. The results would have been probably much more favorable for prompt caching if the context size would have been in the magnitude of tens or hundreds thousands tokens, instead of 7157 used in the tests.

However, long tail latencies of the MuSiQue corpus seems to benefit very well from the prompt caching, indicating that when the computational resources of the LLM provider are scarce, using cached prefixes instead of recalculating it provides substantial benefit. For instance, Small-to-Big and Model cascading had a total tail latency of 14 seconds in Table 5 went down to 4.6 second, when the prompt caching were included. Absolute latency drop was even higher when prompt caching was added on top of S2B + SR + MC configuration, reducing it 26.7 seconds.

One indication of this finding is that enterprises could implement dynamic caching based on the current load of the LLM provider. When there are excess capacity, adding the cache does not provide any meaningful latency benefits, but during the rush hours, it is very useful. Another considerations is the costs savings, that were large when using caching, more deeply analyzed in subsections 5.2.1 and 5.2.2. To conclude, in these testing setups, prompt caching acted purely as cost saving optimization method.

In addition, there are multiple other ways to cache the prefix instead of explicit KV cache. For example, application level semantic caching could be interesting possibility for deeper consideration (see, for example RAGCacheJin et al. [45] or Redis (https://redis.io/docs/latest/develop/ai/redisvl/api/cache/ (is this good?)). Also, all LLM providers have own implementations of caching, and the analysis provided here is only restricted the caching options that Google Vertex AI provides.

6.2 Model Cascading and the Frontier Over-Refusal Paradox

6.3 Multi-Module Interactions: Synergy Bifurcation and Attention Anchoring

6.4 Corpus Topology, the Retrieval Noise Threshold, and Enterprise Deployment

6.5 Methodological Reflections and Limitations

7 Conclusion

References

[1] Stanford University Human-Centered Artificial Intelligence Institute. The Artificial Intelligence Index Report 2026. Technical report, Stanford University, April 2026. URL https://hai.stanford.edu/ai-index/2026-ai-index-report.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[3] Google Cloud. *Gemini Enterprise Agent Platform Pricing: Generative AI and Foundation Models*, 2026. URL https://cloud.google.com/gemini-enterprise-agent-platform/generative-ai/pricing. Accessed September 2026.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[5] Shangyu Wu, Ying Xiong, Yufei Cui, Haolun Wu, Can Chen, Ye Yuan, Lianming Huang, Xue Liu, Tei-Wei Kuo, Nan Guan, and Chun Jason Xue. Retrieval-augmented generation for natural language processing: A survey. *arXiv preprint arXiv:2407.13193*, 2025.

[6] Siyun Zhao, Yuqing Yang, Zilong Wang, Zhiyuan He, Luna K. Qiu, and Lili Qiu. Retrieval augmented generation (RAG) and beyond: A comprehensive survey on how to make your LLMs use external data more wisely. *arXiv preprint arXiv:2409.14924*, 2024.

[7] Broadcom Inc. Spring AI. https://spring.io/projects/spring-ai, 2026. Accessed: 2026-06-24.

[8] Elias Lumer, Matt Melich, Olivia Zino, Elena Kim, Sara Dieter, Pradeep Honaganahalli Basavaraju, Vamse Kumar Subbiah, James A. Burke, and Roberto Hernandez. Rethinking retrieval: From traditional retrieval augmented generation to agentic and non-vector reasoning systems in the financial domain for large language models. *arXiv preprint arXiv:2511.18177*, 2025.

[9] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. RankRAG: Unifying context ranking with retrieval-augmented generation in LLMs. In *38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, 2024.

[10] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of the 5th MLSys Conference*, 2024.

[11] American Bar Association. BC tribunal confirms companies remain liable for information provided by AI chatbot. https://www.americanbar.org/groups/business_law/resources/business-law-today/2024-february/bc-tribunal-confirms-companies-remain-liable-information-provided-ai-chatbot/, February 2024. Accessed: 2026-06-24.

[12] Antti-Jussi Asunmaa. Large language models-based agents in software development. Bachelor's thesis, Aalto University, 2025. URL https://aaltodoc.aalto.fi/items/80839987-d1bc-46ac-8e2f-7f051e8b849e.

[13] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson Education, Inc., Upper Saddle River, New Jersey, third edition, 2010. ISBN 978-0-13-604259-4.

[14] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, et al. A survey of large language models. *Frontiers of Computer Science*, 20(12):2012627, 2026.

[15] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. 2019.

[16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017*, pages 5998–6008, Long Beach, CA, USA, December 4-9 2017. Neural Information Processing Systems (NIPS).

[17] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[18] Tristan Coignion, Clément Quinton, and Romain Rouvoy. A Performance Study of LLM-Generated Code on Leetcode. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, EASE 2024, page 79–89. ACM, June 2024. doi: 10.1145/3661167.3661221.

[19] Numa Dhamani. *Introduction to Generative AI*. Manning Publications Co. LLC, New York, 1st ed. edition, 2024. ISBN 9781638354345.

[20] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12: 157–173, 2024.

[21] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38, 2023.

[22] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. Technical report, Google DeepMind, 2024. URL https://arxiv.org/abs/2403.05530.

[23] Google DeepMind. Gemini 3 Flash Model Card. Model card, Google, December 2025. URL https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf.

[24] Bhuwan Dhingra, Jeremy R Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W Cohen. Time-aware language models as temporal knowledge bases. *Transactions of the Association for Computational Linguistics*, 10:257–273, 2022.

[25] Paulo Roberto de Moura Júnior, Jean Lelong, and Annabelle Blangero. Adaptive chunking: Optimizing chunking-method selection for RAG. *arXiv preprint arXiv:2603.25333*, 2026.

[26] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, Haofen Wang, et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1):32, 2023.

[27] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE transactions on big data*, 7(3):535–547, 2019.

[28] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 3784–3803, 2021.

[29] Bo Ni, Zheyuan Liu, Leyao Wang, Yongjia Lei, Yuying Zhao, Xueqi Cheng, Qingkai Zeng, Luna Dong, Yinglong Xia, Krishnaram Kenthapadi, et al. Towards trustworthy retrieval augmented generation for large language models: A survey. *arXiv preprint arXiv:2502.06872*, 2025.

[30] Claes Wohlin. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th international conference on evaluation and assessment in software engineering*, pages 1–10, 2014.

[31] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. Evaluation of retrieval-augmented generation: A survey. In Wenwu Zhu, Hui Xiong, Xiuzhen Cheng, Lizhen Cui, Zhicheng Dou, Junyu Dong, Shanchen Pang, Li Wang, Lanju Kong, and Zhenxiang Chen, editors, *Big Data*, pages 102–120, Singapore, 2025. Springer Nature Singapore. ISBN 978-981-96-1024-2.

[32] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models

through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, 2024.

[33] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations (ICLR 2025)*, 2025.

[34] Minghao Guo, Qingcheng Zeng, Xujiang Zhao, Yanchi Liu, Wenchao Yu, Mengnan Du, Haifeng Chen, and Wei Cheng. Deepsieve: Information sieving via LLM-as-a-knowledge-router. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 3054–3077, 2026.

[35] Jiarui Zhang, Xiangyu Liu, Yong Hu, Chaoyue Niu, Fan Wu, and Guihai Chen. RAGRouter: Learning to route queries to multiple retrieval-augmented language models. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025.

[36] Jaewan Park, Solbee Cho, and Jay-Yoon Lee. Stop-RAG: Value-based retrieval control for iterative RAG. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025) Workshop: Multi-Turn Interactions in Large Language Models*, 2025.

[37] Zihang Li, Wenjun Liu, Yikun Zong, Jiawen Tao, Siying Dai, Songcheng Ren, Zirui Liu, Yuhang Wang, Yanbing Jiang, and Tong Yang. Bridge-RAG: An abstract bridge tree-based retrieval augmented generation algorithm. *arXiv preprint arXiv:2603.26668*, 2026.

[38] Shengyuan Chen, Chuang Zhou, Zheng Yuan, Qinggang Zhang, Zeyang Cui, Hao Chen, Yilin Xiao, Jiannong Cao, and Xiao Huang. LogicRAG: Retrieval augmented generation with adaptive reasoning structures. In *The Fortieth AAAI Conference on Artificial Intelligence (AAAI-26)*, 2026.

[39] Minsang Kim, Cheoneum Park, and Seungjun Baek. QPaug: Question and passage augmentation for open-domain question answering of LLMs. *arXiv preprint arXiv:2406.14277v2*, 2024.

[40] Mateusz Śmigielski, Michał Rajkowski, Mateusz Zbrocki, Michał Bernacki-Janson, Karol Kunicki, Julianna Godziszewska, Maciej Piasecki, and Konrad Wojtasik. Chunking methods on retrieval-augmented generation – effectiveness evaluation against computational cost and limitations. *arXiv preprint arXiv:2606.00881*, 2026.

[41] Gabriel de Souza P. Moreira, Ronay Ak, Benedikt Schifferer, Mengyao Xu, Radek Osmulski, and Even Oldridge. Enhancing Q&A text retrieval with ranking models: Benchmarking, fine-tuning and deploying rerankers for RAG. *arXiv preprint arXiv:2409.07691*, 2024.

[42] Zhiyuan Cheng, Longying Lai, Yue Liu, Kai Cheng, and Xiaoxi Qi. Enhancing financial report question-answering: A retrieval-augmented generation system with reranking analysis. *arXiv preprint arXiv:2603.16877*, 2026.

[43] Zihan Wang, Zihan Liang, Zhou Shao, Yufei Ma, Huangyu Dai, Ben Chen, Lingtao Mao, Chenyi Lei, Yuqing Ding, and Han Li. InfoGain-RAG: Boosting retrieval-augmented generation through document information gain-based reranking and filtering. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7190–7204. Association for Computational Linguistics, 2025.

[44] Zhepei Wei, Wei-Lin Chen, and Yu Meng. InstructRAG: Instructing retrieval-augmented generation via self-synthesized rationales. In *Published as a conference paper at ICLR 2025*, 2025.

[45] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems (TOCS)*, 44(1):2:1–2:27, 2025.

[46] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6588–6601. Association for Computational Linguistics, 2025.

[47] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys '25)*, pages 95–110. ACM, 2025.

[48] Jiahao Wang, Weiyu Xie, Mingxing Zhang, Boxing Zhang, Jianwei Dong, Yuening Zhu, Chen Lin, Jingqi Tang, Yaochen Han, Zhiyuan Ai, Xianglin Chen, Yongwei Wu, and Congfeng Jiang. From prefix cache to fusion RAG cache: Accelerating LLM inference in retrieval-augmented generation. *Proc. ACM Manag. Data*, 4(1):41:1–41:28, 2026.

[49] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, and Ali Farhadi. Matryoshka representation learning. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 30233–30249. Curran Associates, Inc., 2022. doi: 10.52202/068431-2192. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/c32319f4868da7613d78af9993100e42-Paper-Conference.pdf.

[50] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.

[51] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, 2024.

[52] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.

[53] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Xiong, and Zheng Liu. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv preprint arXiv:2402.03216*, 2024.

[54] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 2012. ISBN 978-3-642-29043-5. doi: 10.1007/978-3-642-29044-2.

[55] Google Cloud. *Vertex AI Search and Ranking API Pricing*, 2026. URL https://cloud.google.com/vertex-ai/generative-ai/pricing#ranking-api. Accessed September 2026.

A Enterprise Knowledge Corpus Extraction Methodology

The enterprise dataset was constructed by crawling the customer-facing public web pages of the partner enterprise. The crawler operates via a breadth-first search (BFS) strategy implemented with the Jsoup library. It was strictly restricted to Finnish-language content for the consumer and retail segment under the `/henkiloasiakkaat/` path, while other language versions and authenticated customer portals were excluded to establish a well-defined evaluation scope. All non-content elements—including navigation menus, cookie banners, tracking scripts, and modal overlays—were systematically removed to minimize retrieval noise. In addition, because realistic customer inquiries frequently concern product terms and conditions (T&C) as well as service guides, linked PDF documents were downloaded and converted to structured Markdown using Apache PDFBox. The resulting raw enterprise corpus comprises exactly 250 web pages and 50 PDF documents. The data ingestion pipeline and dataset scripts are available in the project repository.[4]

Because the academic MuSiQue benchmark follows a strictly structured schema with a question, reference answer, and exactly 20 candidate passages per sample, the enterprise evaluation dataset was constructed to mirror this format to ensure rigorous comparability. The closed enterprise dataset was generated from the raw crawled corpus through the following procedure:

First, candidate document paragraphs were processed from the scraped Markdown files. For factual single-hop and multi-hop queries, 1 or 2 supporting passages containing the necessary evidence were identified. To replicate the retrieval difficulty of MuSiQue, the remaining context was filled with hard negative passages and randomly sampled distractor chunks from the wider corpus until each question contained exactly 20 candidate passages. The order of these 20 passages was randomly shuffled to eliminate positional bias. Google Gemini 3.8 Flash was employed during dataset construction to verify the factual answerability of each candidate pair, decompose multi-hop questions into intermediate reasoning steps, and synthesize canonical reference answers with lexical aliases. In addition, the same model was used to generate 20 out-of-domain queries concerning market competitors and external industry topics to evaluate system abstention. Each out-of-domain query was paired with 20 distracting domain passages and the standardized refusal string.

This structural conversion was motivated by the differing retrieval scopes of the two environments. The MuSiQue benchmark spans 100 questions with 20 candidate passages each (2000 total passages), which resolves to exactly 1688 unique text chunks after deduplication. In comparison, the unconstrained raw enterprise corpus scraped from the 300 source documents contains 2420 unique text chunks across 2535 total passages. Evaluating the raw enterprise corpus directly would introduce a severe confounding variable into retrieval evaluation, as the search space would be

[4] https://github.com/eitsei/dippa

significantly noisier than that of MuSiQue. Following the deduplication and sample formatting procedure, the closed enterprise dataset contains 1333 unique reference paragraphs. When indexed into the vector store with 512-token chunk boundaries, these paragraphs yield 1701 unique text chunks (1725 total passages), establishing comparable retrieval density and task difficulty with the academic benchmark. The empirical evaluation results and test statistics for the unconstrained raw enterprise corpus are provided in Appendix D.

B Operational Cost Catalog and Cloud Tariffs

Table B1: Operational pricing catalog and cloud tariffs for generative, re-ranking, and embedding services (verified as of September 2026). Generative model tariffs reflect standard pay-as-you-go pricing for $\leq$ 200K input tokens, corresponding to all evaluated multi-hop and enterprise workload requests.

Model / Service	Pipeline Role	Prompt ($/1M)	Cached ($/1M)	Output ($/1M)	Auxiliary Unit Rate	Source
Gemini 3.5 Flash	Primary Workhorse (C_0, M_4)	$1.50	$0.15	$9.00	–	[3]
Gemini 3.5 Flash-Lite	Router Classifier (M_4)	$0.30	$0.03	$2.50	–	[3]
Gemini 3.1 Pro Preview	Complex Multi-Hop (M_4)	$2.00	$0.20	$12.00	–	[3]
Vertex AI Ranking	Cross-Encoder Re-rank (M_2)	–	–	–	$0.00010/doc	[55]
Embeddings for Text	Dense Vectorization	$0.025	–	–	–	[3]
Qwen3.8-27B	Evaluation Judge (RAGAS)	*Local Execution*			$0.00/query	[52]
BGE-M3	Evaluation Embedding	*Local Execution*			$0.00/query	[53]

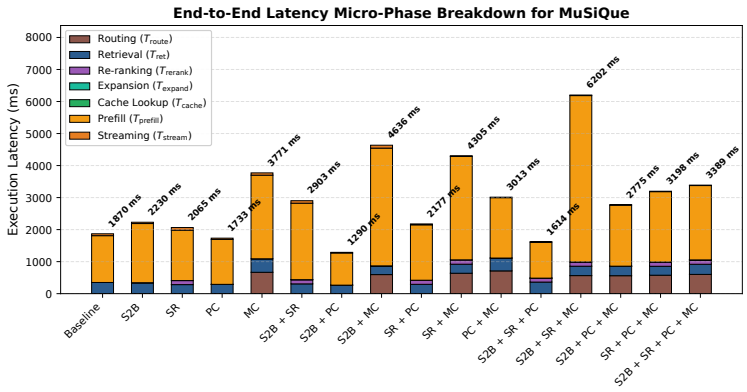
Notes: Generative model rates reflect Google Cloud Gemini Enterprise Agent Platform standard pay-as-you-go list pricing for input windows $\leq$ 200K tokens. Context caching discounts (M_3) provide a 90% unit rate reduction on cached prompt prefixes. Candidate re-ranking (M_2) is billed per scored document ($50 \times \$0.00010 = \$0.0050$/query). RAGAS quality evaluation is conducted entirely offline.

C Detailed Benchmark Performance Profiles and Component Breakdowns

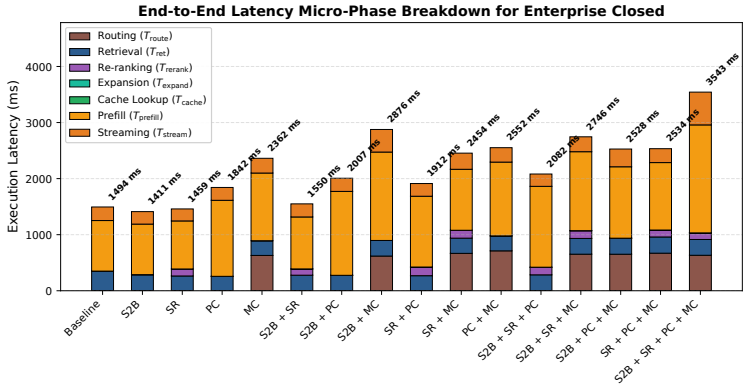
This appendix presents the comprehensive component-level profiling visualizations across all 16 evaluated architectural pipeline configurations on both the academic multi-hop benchmark (MuSiQue) and the closed enterprise production corpus. These detailed distributions complement the summary metric tables and trade-off frontiers presented in Chapter 5.

C.1 Component-Wise Latency Breakdowns

Figure C1 details the empirical execution latency decomposed across fine-grained pipeline stages: dense vector retrieval, cross-encoder neural re-ranking (M_2), prompt caching prefix initialization and reuse (M_3), query classification routing (M_4), and generative token synthesis across both evaluation benchmarks.



(a) Academic multi-hop benchmark (MuSiQue)

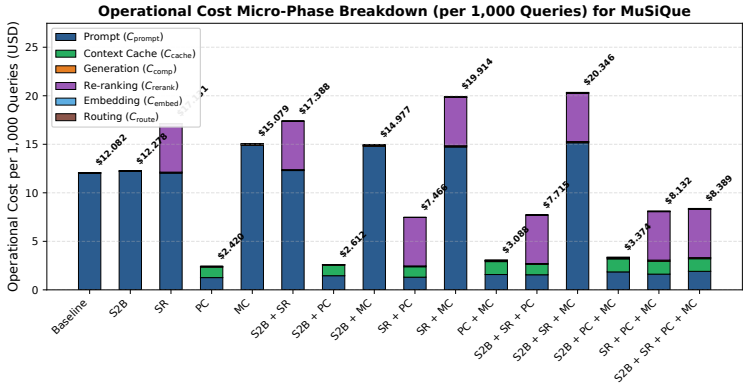


(b) Enterprise closed benchmark corpus

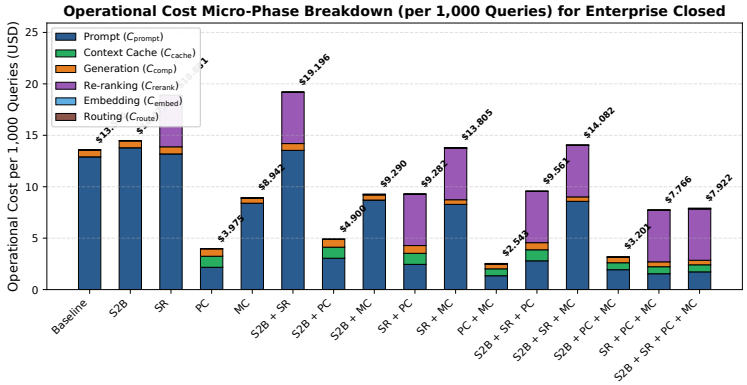
Figure C1: Component-wise execution latency breakdown across all 16 pipeline configurations on (a) MuSiQue and (b) Enterprise Closed, decomposing latency across retrieval, re-ranking, prompt caching, model routing, and token generation phases.

C.2 Operational Expenditure Component Decompositions

Figure C2 presents the component-wise monetary operational expenditure per 1,000 queries, isolating costs incurred from uncached context prompt tokens, context cache read lookups, generative completion tokens, auxiliary cross-encoder neural re-ranking calls, and router model classifications across both benchmark environments.



(a) Academic multi-hop benchmark (MuSiQue)



(b) Enterprise closed benchmark corpus

Figure C2: Component-wise operational expenditure breakdown per 1,000 queries across all 16 pipeline configurations on (a) MuSiQue and (b) Enterprise Closed, decomposing monetary expenditure across prompt prefill, context cache lookups, generative completion, auxiliary neural re-ranking, and model routing.

C.3 Multi-Dimensional Quality Metric Profiles

Figure C3 and Figure C4 illustrate configuration-level response quality profiles across Context Precision, Context Recall, Faithfulness, and Answer Relevancy across architectural tiers ($k = 0 \ldots 4$).

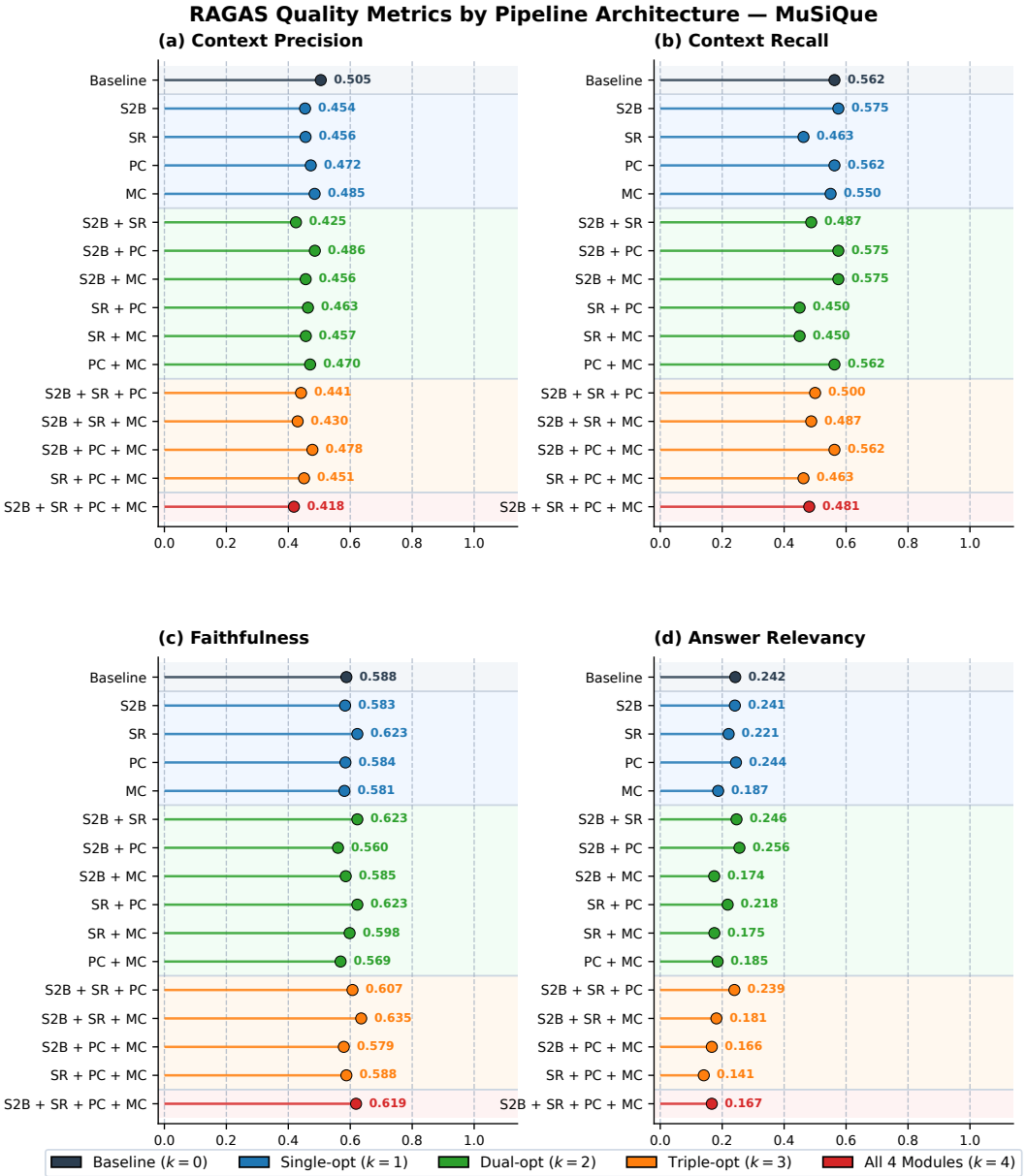


Figure C3: Multi-dimensional response quality profiles across architectural configurations on the academic multi-hop benchmark (MuSiQue), comparing Context Precision, Context Recall, Faithfulness, and Answer Relevancy across architectural tiers ($k = 0 \ldots 4$). Horizontal lollipop plots with tier-stratified background shading isolate configuration-level variations across retrieval and synthesis dimensions.

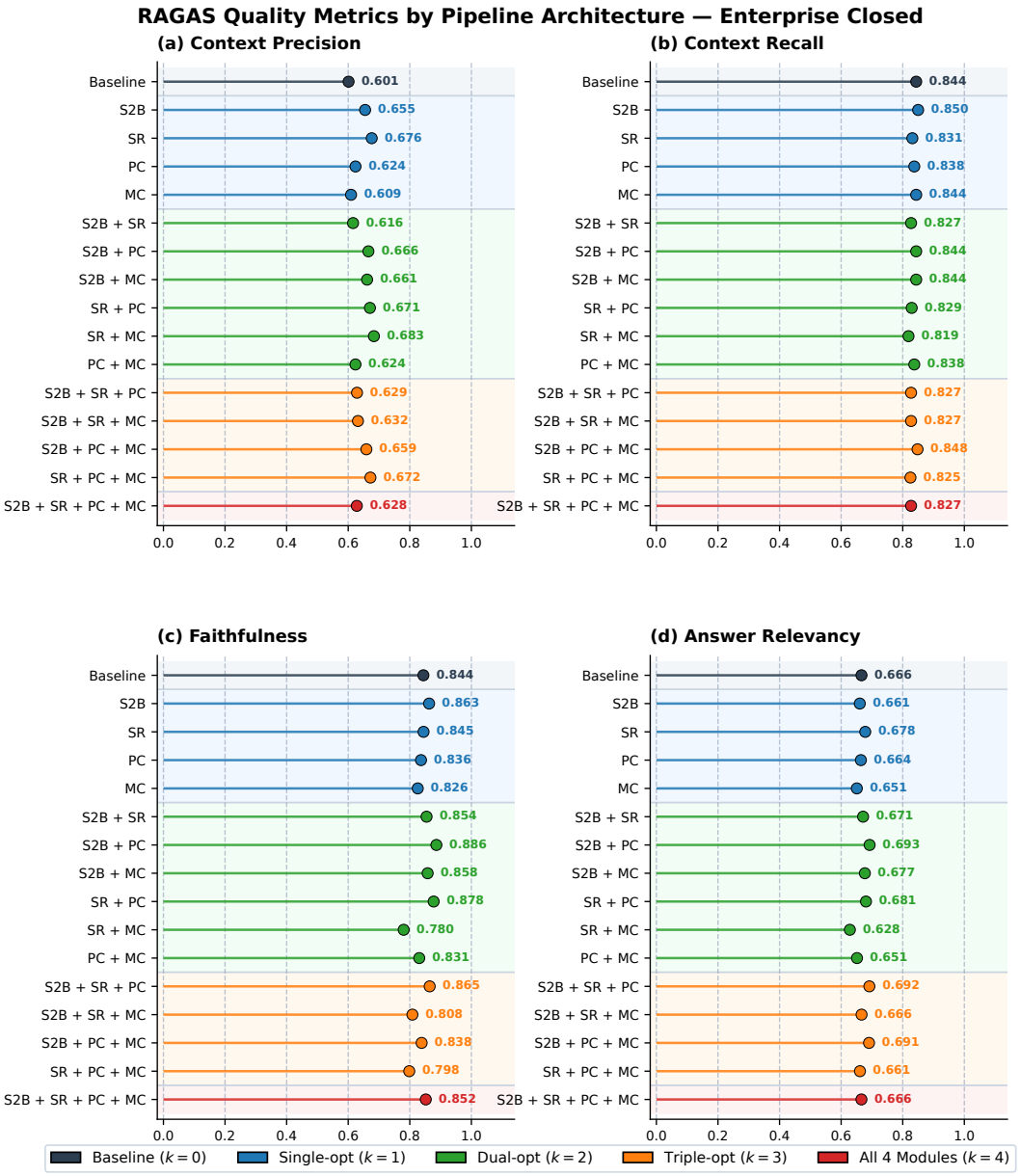


Figure C4: Multi-dimensional response quality profiles across architectural configurations on the enterprise closed benchmark corpus, comparing Context Precision, Context Recall, Faithfulness, and Answer Relevancy across architectural tiers ($k = 0 \ldots 4$). Horizontal lollipop plots with tier-stratified background shading isolate configuration-level variations across retrieval and synthesis dimensions.

D Supplementary Benchmark Evaluation on Unconstrained Raw Enterprise Corpus

This appendix presents the comprehensive benchmark evaluation statistics across all 16 evaluated architectural pipeline configurations on the original, unconstrained raw enterprise knowledge corpus.

Table D1: End-to-end execution latency and Time-to-First-Token (TTFT) across 16 pipeline architectures on the unconstrained raw Enterprise corpus ($N = 100$). Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline.

Pipeline	**TTFT** P_{50} (ms)	**TTFT** P_{95} (ms)	$T_{\text{total}} P_{50}$ (ms)	$T_{\text{total}} P_{95}$ (ms)	ΔT_{total} (%)
Baseline ($k = 0$)					
Naive RAG Baseline	1438.0	1780.0	1660.5	2320.2	–
Single-Factor Isolated ($k = 1$)					
S2B	1421.0	1779.6	1621.0	2257.3	-2.4%^{ns}
SR	1616.0	2033.0	1802.5	2646.2	+8.6%***
PC	1694.5	2753.0	1969.0	3886.6	+18.6%***
MC	2008.5	11846.9	2208.5	11846.9	+33.0%***
Pairwise Combinations ($k = 2$)					
S2B + SR	1764.5	3456.9	2020.0	3554.6	+21.7%***
S2B + PC	1520.0	2028.0	1682.5	2550.9	+1.3%^{ns}
S2B + MC	1850.5	3337.0	2112.0	4016.8	+27.2%***
SR + PC	1615.0	2020.8	1762.0	2648.4	+6.1%***
SR + MC	2125.5	4495.5	2339.5	5007.1	+40.9%***
PC + MC	1940.5	3357.5	2159.5	3758.8	+30.1%***
Higher-Order Combinations ($k \geq 3$)					
S2B + SR + PC	1911.5	2875.8	2155.5	3093.9	+29.8%***
S2B + SR + MC	2246.0	4599.8	2444.5	4992.2	+47.2%***
S2B + PC + MC	2061.5	4046.6	2301.0	4447.7	+38.6%***
SR + PC + MC	2267.5	3967.1	2517.5	4523.6	+51.6%***
S2B + SR + PC + MC	2254.0	4083.2	2510.0	4730.5	+51.2%***

Notes: P_{50} and P_{95} denote median and 95th percentile execution latencies. ΔT_{total} denotes percentage change relative to baseline. $^{*}p < 0.05, ^{**}p < 0.01, ^{***}p < 0.001, ^{\text{ns}}p \geq 0.05$ (paired Wilcoxon with Holm-Bonferroni correction vs. naive baseline).

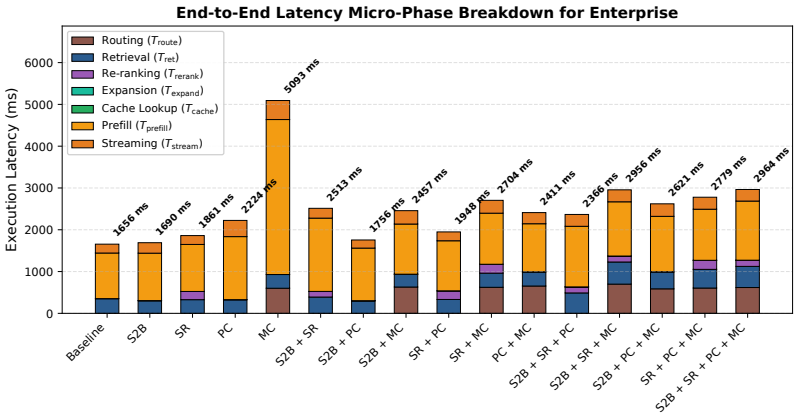


Figure D1: Component-wise latency breakdown across all 16 pipeline configurations on the unconstrained raw enterprise corpus, detailing execution duration across retrieval, semantic re-ranking, cache management, and model tier generation phases.

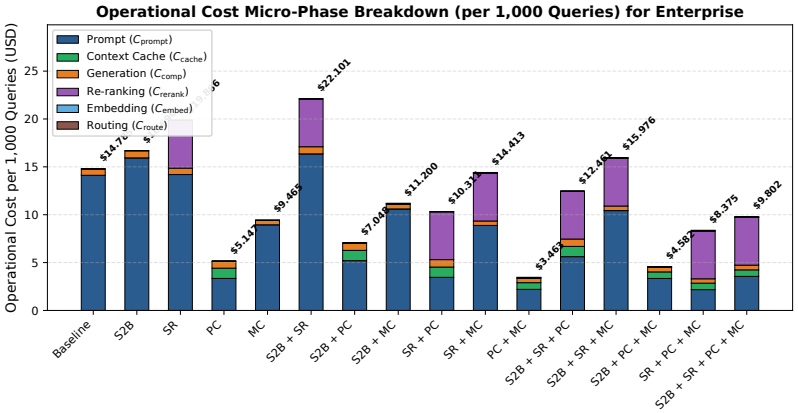


Figure D2: Component-wise operational expenditure breakdown per 1,000 queries across all 16 pipeline configurations on the unconstrained raw enterprise corpus, detailing monetary expenditure across uncached context prefill, context cache lookups, generative completion, and auxiliary neural re-ranking services.

Table D2: Operational cost and token consumption breakdown across 16 pipeline architectures on the unconstrained raw Enterprise corpus ($N = 100$). Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline.

Pipeline	Uncached Prompt Tok	Cached Prompt Tok	Output Tokens	Cost / 1k ($)	Δ Cost (%)
Baseline ($k = 0$)					
Naive RAG Baseline	9,412	0	74	$14.785	–
Single-Factor Isolated ($k = 1$)					
S2B	10,627	0	83	$16.690	+12.9%***
SR	9,465	0	74	$19.866	+34.4%***
PC	2,230	7,157	81	$5.147	-65.2%***
MC	9,412	0	66	$9.465	-36.0%***
Pairwise Combinations ($k = 2$)					
S2B + SR	10,904	0	83	$22.101	+49.5%***
S2B + PC	3,468	7,157	86	$7.048	-52.3%***
S2B + MC	10,627	0	73	$11.200	-24.3%***
SR + PC	2,306	7,157	86	$10.311	-30.3%***
SR + MC	9,465	0	70	$14.413	-2.5%*
PC + MC	2,253	7,157	73	$3.463	-76.6%***
Higher-Order Combinations ($k \geq 3$)					
S2B + SR + PC	3,745	7,157	86	$12.461	-15.7%***
S2B + SR + MC	10,904	0	67	$15.976	+8.1%**
S2B + PC + MC	3,468	7,157	71	$4.582	-69.0%***
SR + PC + MC	2,306	7,157	71	$8.375	-43.4%***
S2B + SR + PC + MC	3,745	7,157	73	$9.802	-33.7%***

Notes: Per-query sample token averages. Cost normalized per 1,000 queries based on Google Cloud Gemini enterprise tariffs. ΔCost denotes percentage change relative to baseline. $^{*}p < 0.05$,$^{**}p < 0.01$,$^{***}p < 0.001$, $^{\text{ns}}p \geq 0.05$ (paired Wilcoxon with Holm-Bonferroni correction vs. naive baseline).

Table D3: Retrieval, generation, and out-of-domain rejection quality metrics across 16 pipeline architectures on the unconstrained raw Enterprise Knowledge Corpus ($N = 100$). Statistical significance flags denote Holm-Bonferroni-adjusted paired Wilcoxon signed-rank tests against the naive baseline.

Pipeline	Context Precision	Context Recall	Faithfulness	Answer Relevancy	Refusal Acc. / FRR (%)	Composite Quality Q
Baseline ($k = 0$)						
Naive RAG Baseline	0.564	0.744	0.756	0.620	100.0% / 18.8%	0.5022
Single-Factor Isolated ($k = 1$)						
S2B	0.687	0.785	0.856	0.683	100.0% / 11.2%	0.6368^{ns}
SR	0.646	0.742	0.807	0.645	100.0% / 15.0%	0.5479^{ns}
PC	0.562	0.744	0.784	0.624	100.0% / 17.5%	0.5326^{ns}
MC	0.558	0.750	0.759	0.630	100.0% / 18.8%	0.5149^{ns}
Pairwise Combinations ($k = 2$)						
S2B + SR	0.718	0.794	0.870	0.720	100.0% / 7.5%	0.6433^{**}
S2B + PC	0.707	0.792	0.855	0.695	100.0% / 8.8%	$\mathbf{0.6633}^{\text{ns}}$
S2B + MC	0.703	0.792	0.834	0.662	100.0% / 12.5%	0.6492^{ns}
SR + PC	0.656	0.748	0.853	0.683	100.0% / 11.2%	0.5688^{ns}
SR + MC	0.631	0.760	0.817	0.649	100.0% / 15.0%	0.5596^{ns}
PC + MC	0.566	0.744	0.832	0.648	100.0% / 15.0%	0.5634^{ns}
Higher-Order Combinations ($k \geq 3$)						
S2B + SR + PC	0.708	0.787	0.890	0.716	100.0% / 7.5%	0.6466^{**}
S2B + SR + MC	0.682	0.781	0.849	0.665	100.0% / 12.5%	0.6162^{*}
S2B + PC + MC	0.688	0.798	0.804	0.650	95.0% / 15.0%	0.6268^{ns}
SR + PC + MC	0.654	0.742	0.782	0.633	100.0% / 16.2%	0.5434^{ns}
S2B + SR + PC + MC	0.680	0.785	0.833	0.677	100.0% / 12.5%	0.6028^{ns}

Notes: Evaluated via RAGAS [51] on answerable queries. Refusal Acc. denotes abstention on unanswerable out-of-domain (OOD) queries; FRR measures false refusals on answerable queries. Composite Quality $Q \in [0, 1]$ represents regularized harmonic mean (Eq. 20). Best Q in bold. $^{*}p < 0.05$,$^{**}p < 0.01$,$^{***}p < 0.001$, $^{\text{ns}}p \geq 0.05$ (paired Wilcoxon with Holm-Bonferroni correction vs. naive baseline).

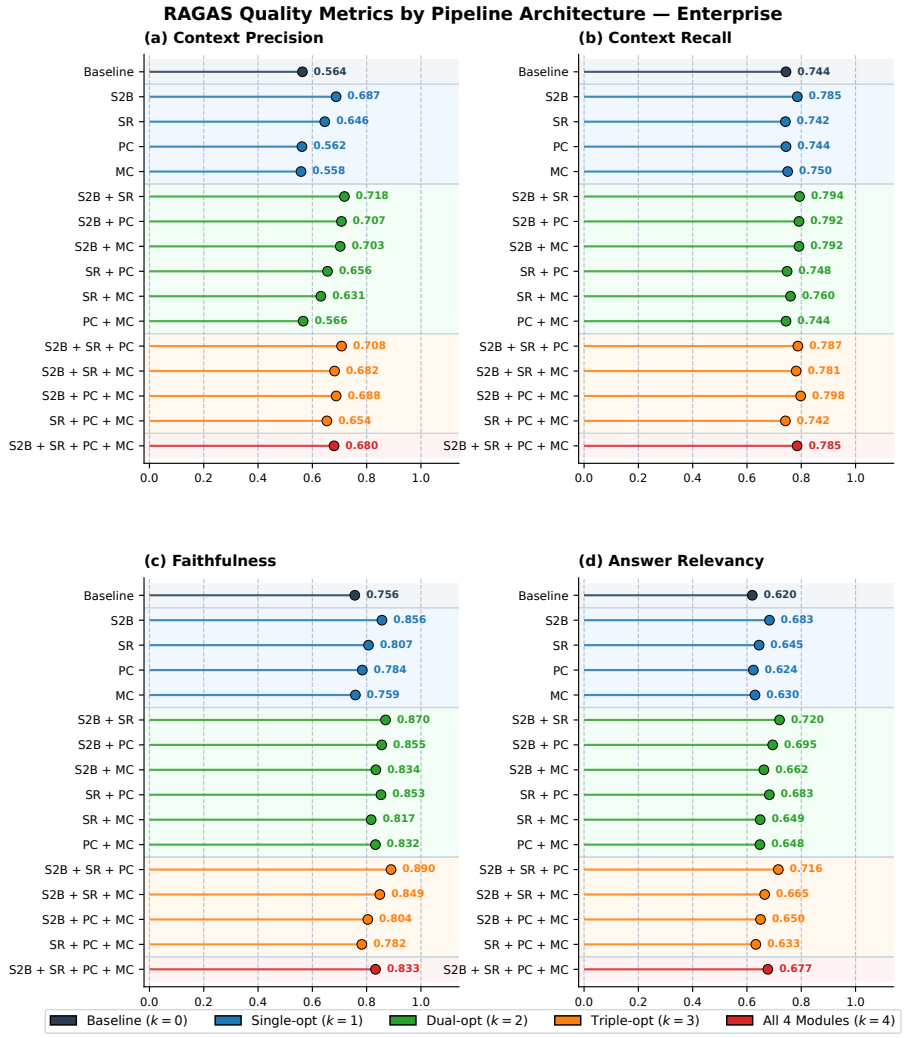


Figure D3: Multi-dimensional response quality profiles across architectural configurations on the unconstrained raw enterprise corpus, comparing Context Precision, Context Recall, Faithfulness, and Answer Relevancy across architectural tiers ($k = 0 \ldots 4$). Horizontal lollipop plots with tier-stratified background shading isolate configuration-level variations.

Table D4: Multi-objective Pareto efficiency and 3D non-domination trade-off metrics across 16 pipeline architectures on the unconstrained raw Enterprise corpus ($N = 100$).

Pipeline	Composite Quality Q	Cost / 1k ($)	$T_{\text{total}} P_{50}$ (s)	QPD	QPS	Pareto Optimal
Baseline ($k = 0$)						
Naive RAG Baseline	0.5022	$14.785	1.66	34	0.30	**Yes**
Single-Factor Isolated ($k = 1$)						
S2B	0.6368	$16.690	1.62	38	0.39	**Yes**
SR	0.5479	$19.866	1.80	28	0.30	No
PC	0.5326	$5.147	1.97	103	0.27	**Yes**
MC	0.5149	$9.465	2.21	54	0.23	No
Pairwise Combinations ($k = 2$)						
S2B + SR	0.6433	$22.101	2.02	29	0.32	No
S2B + PC	0.6633	$7.048	1.68	94	0.39	**Yes**
S2B + MC	0.6492	$11.200	2.11	58	0.31	No
SR + PC	0.5688	$10.311	1.76	55	0.32	No
SR + MC	0.5596	$14.413	2.34	39	0.24	No
PC + MC	0.5634	$3.463	2.16	163	0.26	**Yes**
Higher-Order Combinations ($k \geq 3$)						
S2B + SR + PC	0.6466	$12.461	2.16	52	0.30	No
S2B + SR + MC	0.6162	$15.976	2.44	39	0.25	No
S2B + PC + MC	0.6268	$4.582	2.30	137	0.27	**Yes**
SR + PC + MC	0.5434	$8.375	2.52	65	0.22	No
S2B + SR + PC + MC	0.6028	$9.802	2.51	61	0.24	No

Notes: Composite Quality $Q \in [0, 1]$ (Eq. 20). QPD = Q/Cost_{1k} and QPS = Q/T_{total} represent resource efficiency ratios. Pareto optimality denotes 3D non-domination (max Q, min Cost, min T_{total}). Bold indicates Pareto-optimal configuration.

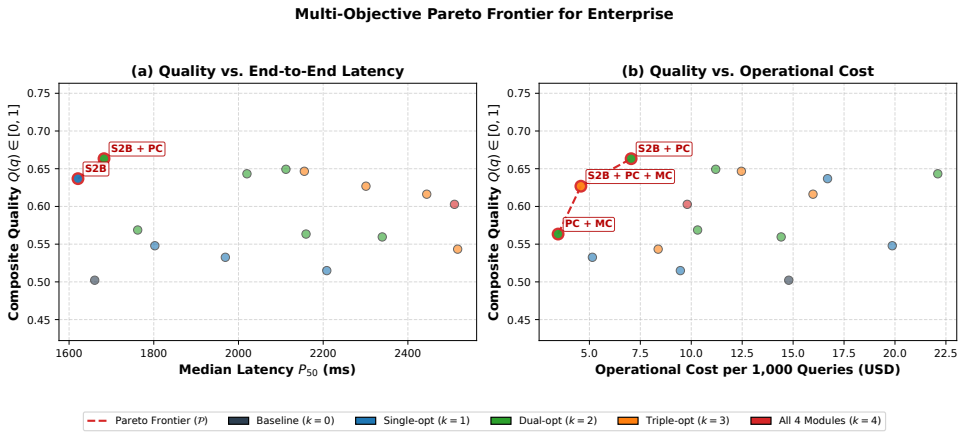


Figure D4: Empirical Pareto frontier of Composite Quality (Q) versus median end-to-end execution latency (T_{total} P_{50}) and operational expenditure (Cost_{1k}) on the unconstrained raw enterprise corpus. Solid lines delineate the non-dominated empirical Pareto frontier across configurations.